Bayesian POMDP Controllers for LLM Critic Stack Orchestration

Research progress & experimental results

MBZUAI

May 7, 2026

Outline

Click any item to jump to that section. Colors: **blue**=core conceptual blocks, **green**=positive findings, **orange**=notable / has caveat, **red**=limitations, teal=cross-reference.

What we're doing — one-sentence version

Research question

Given an LLM that proposes a code patch and a stack of cheap-to-expensive critics (syntax check, lint, public tests, LLM review), **when should we verify a patch with a costly oracle vs. regenerate?**

Why this matters:

- **Verification is the dominant cost** — running full test suites, calling frontier LLMs as judges, or paying for human review.
- **Most pipelines use rigid heuristics:** "always verify," "best-of-N," "run all critics." **They ignore the posterior probability** that the patch is correct given cheap signals.
- **A principled answer is a partially-observable MDP:** state = $P(Y{=}1 \mid \text{evidence})$, actions $\in \{\text{run critic } i, \text{verify, stop, regen}\}$, solve via dynamic programming.

Concrete setup

Per LLM patch (instance i, attempt k):

1. Critics produce noisy binary signals $L_0, L_1, L_2, L_3 \in \{0, 1\}$ (syntax, lint, public-tests, LLM-review).
2. Oracle verifier produces ground truth $Y \in \{0, 1\}$ (full test suite).
3. Each critic has a known cost c_0, c_1, c_2, c_3, oracle cost c_v.
4. A reward r is paid for $Y{=}1$.

Optimal policy maximises $\mathbb{E}[r \cdot Y - \text{costs}]$ over policies π : history $\to$ action.

Two regimes our framework distinguishes

- **Inference-driven:** cheap critics shift the posterior enough to skip the oracle on confident negatives. Bayesian controller wins.
- **Oracle-cheap:** verifying is so cheap that always-verify is indistinguishable. Bayesian gain ≈ 0.

Baselines we compare against

Policy	What it does
`always_verify`	Run oracle on every first patch.
`best_of_N`	Generate *N* patches, verify last (or vote).
`threshold_L0`	Run cheap syntax check. Verify iff PASS. Else regen.
`threshold_L2`	Run public tests. Verify iff PASS. Else regen. (LCB-only)
`threshold_L3`	Run LLM-judge review. Verify iff PASS. Else regen.
`fixed_pipeline`	Run *all* cheap critics, verify iff all PASS.
Iterative refinement methods	
`Self-Refine` (Madaan 2023)	Generate → self-critique → refine. Stops on "CRITIQUE_OK."
`Reflexion` (Shinn 2023)	Generate → external test → reflect → refine. Stops on test pass.

First 6: stateless per-patch policies. Self-Refine and Reflexion are stateful trajectories we treat as competing methods.

Our methods

Method	What it does
Bayesian belief-state controllers (operate on $b_t = P(Y_t{=}1 \mid \text{history})$)	
`bayesian_greedy`	1-step lookahead: at each state, pick `argmax` of immediate Q-value. Doesn't use the kernel; treats one critic + verify as the full plan.
`bayesian_DP`	Backward induction on discretised belief grid (51 pts $\times$ horizon $H{=}3$). Uses the *transition kernel* $P(Y_{k+1} \mid Y_k)$; can chain critics multi-step.

Shared by both:

- Same belief update $b' = \frac{P(z|Y{=}1)\, b}{P(z|Y{=}1)\, b + P(z|Y{=}0)(1-b)}$
- Same action set $\{\text{verify}, \text{generate}, \text{give\_up}, L_0, L_2, L_3\}$
- Same cost vector $(c_{\text{gen}}, c_{\text{ver}}, c_{L_0}, c_{L_2}, c_{L_3})$

Differ on:

- Planning horizon: 1 vs $H{=}3$
- Uses kernel for planning? No vs Yes
- Can chain critics? No vs Yes
- Robust to kernel misspecification? More so vs less

Code: `GreedyController` in `lcb_compare.py:52` `BayesianController` (DP) in `run_baseline_vs_controller.py:68`.

`bayesian_greedy` — 1-step lookahead

Mechanism: at every state (b, k), evaluate each action's *immediate* Q-value assuming the very next move is `verify`. **Does not use the transition kernel** — by design, Q_{generate} substitutes the prior for the post-generate belief.

What data this method needs

Calibration corpus only: prior $P(Y{=}1)$ + critic likelihoods $P(z \mid Y)$.
No iter / no kernel needed for setup. Iter trajectories are only used later, at evaluation, to simulate the policy and compute utility.
+ a cost vector $(c_{\text{gen}}, c_{\text{ver}}, c_{L_0}, c_{L_2}, c_{L_3})$.

Q-values (myopic, 1-step horizon)

$$Q_{\text{verify}} = R \cdot b - c_{\text{ver}}$$

$$Q_{\text{generate}} = -c_{\text{gen}} + R \cdot \text{prior} - c_{\text{ver}} \quad \text{(uses prior, NOT kernel)}$$

$$Q_{L_k} = -c_k + \mathbb{E}_z\left[\max(R \cdot b' - c_{\text{ver}},\ 0)\right]$$

Worked example — LCB-medium / haiku-4.5 (prior 0.34, L_2 gap 0.92, $R{=}100$)

Initial $b_0 = 0.34$: $Q_{\text{verify}}{=}\mathbf{33.0}$, $Q_{L_2}{=}\mathbf{37.3}$, $Q_{\text{give\_up}}{=}0$.
$\Rightarrow$ **argmax** $= L_2$. Observe $z_2{=}$PASS $\Rightarrow b_1 \approx 0.97 \Rightarrow$ next step picks **verify**.

bayesian_DP — backward-induction controller

Mechanism: discretise belief into 51 grid points × horizon H=3; solve Bellman backwards; act greedily w.r.t. the resulting V-table. **Uses the measured kernel** $P(Y_{k+1} \mid Y_k)$ for `generate`; recurses on critic outcomes (can plan L0→L2→`verify` multi-step).

What data this method needs

Calibration corpus → prior + likelihoods (same as greedy). **Iter corpus** (multi-step Self-Refine / Reflexion) → measured kernel via `compute_transition_kernel.py`, giving $P(\text{fix} \mid \text{broken})$, $P(\text{break} \mid \text{correct})$ from $Y_k \to Y_{k+1}$ pairs.

Bellman backward induction (build once per cell)

$$V_H(b) = \max(Rb - c_{\text{ver}},\ 0); \quad Q_k(a;b) = \begin{cases} Rb - c_{\text{ver}} & a{=}\text{verify} \\ -c_{\text{gen}} + V_{k+1}\big(\text{kernel}(b)\big) & a{=}\text{generate} \\ -c_a + \mathbb{E}_z\big[V_k^{\text{post}}(b'_z)\big] & a{=}L_0, L_2, L_3 \\ 0 & a{=}\text{give\_up} \end{cases}$$

Worked example — LCB-easy / gpt-5-mini (P_{fix}=0.53, P_{break}=0.04)

b_0=0.22. **Greedy** substitutes prior ⇒ $Q_{\text{generate}} = 20$. **DP** applies kernel: $\text{kernel}(b_0) = 0.22{\cdot}0.96 + 0.78{\cdot}0.53 = \mathbf{0.625}$, $Q_{\text{generate}} = -1 + V_1(0.625) = \mathbf{59.4}$ ⇒ **generate** → L_2 at b_1=0.625 → **verify** at $b_2 \approx 0.99$.

`BayesianController._build()` in `run_baseline_vs_controller.py:68--210`. Decision: $O(1)$ table lookup. Build: $O(H \cdot 51 \cdot |A|)$ per cell.

Self-Refine vs Reflexion — algorithmically

Self-Refine (no external feedback)

```
patch_0 = generate(prompt)
for k = 1..N:
    critique = model.self_critique(patch_{k-1})
    if "CRITIQUE_OK" in critique:
        return patch_{k-1}
    patch_k = model.refine(patch_{k-1}, critique)
return patch_N
```

Reflexion (external test signal)

```
memory = []
patch_0 = generate(prompt)
for k = 1..N:
    pass, trace = external_test(patch_{k-1})
    if pass: return patch_{k-1}
    reflection = model.reflect(patch, trace, memory)
    memory.append(reflection)
    patch_k = model.refine(patch, memory)
return patch_N
```

Example: gpt5_mini on astropy__astropy-14182 — clean

Problem (excerpt): *add support for the `header_rows` argument in the RST writer to allow customising header rows.*

LLM patch (clean SEARCH/REPLACE format):

```
--- a/astropy/io/ascii/rst.py
+++ b/astropy/io/ascii/rst.py
@@ -57,10 +57,15 @@
     data_class = SimpleRSTData
     header_class = SimpleRSTHeader

-    def __init__(self):
+    def __init__(self, header_rows=None):
         super().__init__(delimi...
```

Critic verdicts on this patch:

- L0_syntax (ast.parse): **PASS**
- L1_lint (ruff): **PASS**
- L3_llm_review (Haiku-4.5): **PASS**
- Y (full test suite): **1 (resolved)**

All critics correctly flagged this as a likely-correct patch.

Example: haiku45 on same task — hallucination

Same problem as before.
LLM patch:

```
--- a/astropy/io/ascii/rst.py
+++ b/astropy/io/ascii/rst.py
@@ -57,10 +57,15 @@
     data_class = SimpleRSTData
     header_class = SimpleRSTHeader

-    def __init__(self, header_rows=None):           # <-- HALLUCINATED!
+    def __init__(self, header_rows=None):
         super().__init__(delimi...
```

Issue: the SEARCH block claims the original code already had `header_rows=None`. Real source has `def __init__(self):`. Patch fails to apply.

Critic verdicts:

- `L0_syntax`: **PASS** (the diff is syntactically valid Python)
- `L1_lint`: **PASS**
- `L3_llm_review`: **PASS** (judge can't see real source)
- `Y`: **0 (unresolved — patch didn't apply!)**

All cheap critics fooled. This is why we need an oracle in the loop.

Example: sonnet45 on `django__django-11620` — format quirk

LLM output (single-bracket header instead of triple —/+++ format):

```
[django/db/models/sql/query.py]
<<<<<<< SEARCH
def deferred_to_data(self, target, callback):
    ...
=======
def deferred_to_data(self, target, callback):
    callback(target, self)
    for ...
>>>>>>> REPLACE
```

Issue: format works via our XML-style parser fallback but is non-canonical. This is why we landed an **XML-style parser + fuzzy SEARCH/REPLACE matcher** (commit `c10eb8a`): handles

- placeholder paths (`[file.py]`) on the line before the diff
- indent-tolerant SEARCH/REPLACE (whitespace doesn't have to be byte-identical)
- triple-bracket and double-bracket variants

Validation: v3 parser pass rate from ∼60% (canonical only) to **∼93%** (canonical + tolerant) on the spot-check $n = 50$ corpus.

Three quantities are needed for the controller

1. Prior $P(Y{=}1)$

Per-(generator, benchmark, difficulty) cell, fraction of step-0 patches that *would* resolve the bug under the oracle.

2. Critic likelihoods $P(\text{critic} = z \mid Y)$

For each critic $\ell \in \{L_0, L_1, L_2, L_3\}$, the conditional pass rate under both ground-truth values. Estimated with Beta(1,1) Laplace smoothing on the calibration corpus.

3. Transition kernel $P(Y_{k+1} \mid Y_k)$

$P(\text{fix} \mid \text{broken})$, $P(\text{break} \mid \text{correct})$ across iterative refinement steps. Measured directly from the iter trajectory data — not assumed.

Two modes of data collection: calibration vs iter

We collect data in two distinct modes, each producing a different kind of evidence about the (generator, benchmark) cell. Both are needed.

Mode 1: Calibration mode (single-shot, independent attempts)

- Sample n instances; for each, generate $k = 3$ *independent fresh* patches.
- Per patch: run all critics + harness oracle to get Y.
- Captures the *generation distribution*: "how good is this LLM at this benchmark, fresh attempt, no feedback?"

Used to estimate quantities that require an unconditional sample: **prior** $P(Y{=}1)$, **critic likelihoods** $P(z \mid Y)$.

Mode 2: **Iter mode** (sequential trajectories with feedback)

- Same n instances; for each, run a Self-Refine or Reflexion trajectory of up to 5 steps.
- Step $k+1$ is *conditioned on* step k's output via the controller's feedback signal.

Two corpora, each for what it cleanly identifies

Corpus	Size per cell	Used for
Calibration (single-shot)	30 instances × 3 fresh patches = 90 step-0 patches	**Prior** $P(Y{=}1)$ **Critic likelihoods** $P(z \mid Y)$
Iterative trajectories (Self-Refine, Reflexion)	30 instances × up to 5 steps ≈ 150 step-records	**Transition kernel** $P(Y_{k+1} \mid Y_k)$

Each parameter is estimated from the source where it's cleanly identifiable.

At decision time, the controller uses all three simultaneously:

- **Prior** seeds belief b_0 at step 0
- **Likelihoods** update b as critic outcomes are observed
- **Kernel** propagates b across refinement steps

Standard partially-observable MDP setup. Same pattern as forecasting: unconditional climatology + observation likelihoods + transition dynamics.

Why single-shot data for prior + likelihoods

The selection-bias trap:

- Step 1+ patches in iter trajectories are *only* drawn from instances where step 0 failed (Self-Refine/Reflexion continued because the proxy stop condition wasn't met).
- Including step 1+ patches when estimating the prior would **systematically bias it downward** — those are conditioned on prior failure, not a fresh attempt.
- Same argument for critic likelihoods: $P(z \mid Y)$ for a multi-step patch is not the same distribution as for a fresh step-0 patch.

Single-shot calibration gives a clean unconditional measurement of the *generation* distribution.

Forecasting analogy

Climatology measures unconditional $P(\text{rain})$ from independent single-day observations. You don't measure it from conditional rates at later forecast horizons — those depend on what already happened. Same logic here.

Honest caveat (also in §6): should likelihoods incorporate step 1+ patches for more data? It's a 3× data gain

Recipe 1a: prior $P(Y=1)$ — data collection

Goal: per (generator, benchmark, difficulty) cell, the fraction of step-0 patches that resolve the bug under the oracle.

Steps 1–3 (collecting raw Y values):

1. **Sample $n = 30$ instances** from the benchmark with `seed=42`. Same instance set across all generators for reproducibility.
2. **For each instance, generate $k = 3$ candidate patches** via the LLM (temperature 0.7, distinct system messages):
   - OpenRouter API for closed-API generators
   - local vLLM endpoint for Qwen2.5-Coder-32B

   Output → `predictions_p{0,1,2}.jsonl` (∼90 patches/cell).
3. **Run harness eval** on every patch:
   - LCB: hidden-test runner returns pass/fail
   - SWE: podman container, applies patch, runs FAIL_TO_PASS + PASS_TO_PASS test sets
   - MBPP+/HumanEval+: assert-based unit tests

   Record $Y_i \in \{0, 1\}$ per patch into `critic_results.jsonl`.

Recipe 1b: prior $P(Y{=}1)$ — estimator & CI

Steps 4–6 (computing the prior):

4. **Compute the point estimator** (sample mean):

$$\hat{P}(Y{=}1) = \frac{1}{N}\sum_{i=1}^{N} Y_i, \quad N = \#\{\text{patches with non-empty diff}\}$$

5. **Wilson 95% confidence interval** on the proportion (better than normal-approx for small N near 0/1 boundaries):

$$\mathrm{CI}_{95\%} = \frac{\hat{p} + \frac{z^2}{2N} \pm z\sqrt{\frac{\hat{p}(1-\hat{p})}{N} + \frac{z^2}{4N^2}}}{1 + \frac{z^2}{N}}, \quad z = 1.96$$

6. **Persist** to `likelihood_tables.json`: {`"prior_Y1"`: $\hat{p}$, `"prior_Y1_ci"`: `[lo, hi]`, `"n"`: `N`}

Concrete example

LCB-hard / gpt5_mini: 18 of 87 non-empty patches resolved
$\Rightarrow \hat{P}(Y{=}1) = \mathbf{0.21}$, Wilson CI **[0.13, 0.31]**

Recipe 2a: critic likelihoods $P(z \mid Y)$ — verdicts

Goal: $P(\text{critic } \ell \text{ passes} \mid Y{=}1)$ vs. $P(\text{passes} \mid Y{=}0)$ for $\ell \in \{L_0, L_1, L_2, L_3\}$.

Notation: $z_\ell \in \{0, 1\}$ is the binary verdict of critic ℓ on a patch:

- $z_\ell = 1 \Leftrightarrow$ critic outputs **PASS** (claims patch is correct).
- $z_\ell = 0 \Leftrightarrow$ critic outputs **FAIL** (claims patch is incorrect).

$Y \in \{0, 1\}$ is the oracle ground truth from the harness.

Steps 1–3 (per-patch verdicts):

1. **Reuse the same 90 patches** from Recipe 1, with Y already known.
2. **Compute each critic's verdict per patch:**
   - L_0 **syntax**: `ast.parse(modified_file)` for each `.py` touched by the diff → `True` iff all parse cleanly.
   - L_1 **lint**: `ruff check` on the modified files → `True` iff zero diagnostics.
   - L_2 **public tests** (LCB-only): run the public examples shipped with the problem → `True` iff all pass.
   - L_3 **LLM review**: prompt Haiku-4.5 with diff + problem statement, parse one-token `PASS`/`FAIL` response.
3. **Build 2×2 confusion matrix** per critic against ground truth Y:

$$\mathrm{TP}_\ell = \#\{z_\ell{=}1 \wedge Y{=}1\}, \quad \mathrm{FN}_\ell = \#\{z_\ell{=}0 \wedge Y{=}1\}$$

$$\mathrm{FP}_\ell = \#\{z_\ell{=}1 \wedge Y{=}0\}, \quad \mathrm{TN}_\ell = \#\{z_\ell{=}0 \wedge Y{=}0\}$$

Recipe 2b: critic likelihoods $P(z \mid Y)$ — smoothing & tests

Steps 4–7 (estimating + validating):

4. **Beta(1,1) Laplace smoothing** (avoids 0/1 estimates with small N):

$$\hat{P}(z_\ell{=}1 \mid Y{=}1) = \frac{\mathrm{TP}_\ell + 1}{\mathrm{TP}_\ell + \mathrm{FN}_\ell + 2}, \quad \hat{P}(z_\ell{=}1 \mid Y{=}0) = \frac{\mathrm{FP}_\ell + 1}{\mathrm{FP}_\ell + \mathrm{TN}_\ell + 2}$$

5. **Compute the gap** (per-critic informativeness):

$$\Delta_\ell = \hat{P}(z_\ell{=}1 \mid Y{=}1) - \hat{P}(z_\ell{=}1 \mid Y{=}0)$$

   Higher gap ⇒ more informative critic.

6. χ^2 **critic-independence test:** verify $L_0 \perp L_1 \perp L_3 \mid Y$ assumption (we assume *product* likelihood for the Bayesian update):

$$\chi^2 = \sum_{\text{cells}} \frac{(O_{ij} - E_{ij})^2}{E_{ij}}$$

   $p > 0.05$ **on all cells** ⇒ independence assumption safe.

7. **Persist** to `likelihood_tables.json`: per-critic {TP, FN, FP, TN, $\hat{P}(\cdot|Y{=}1)$, $\hat{P}(\cdot|Y{=}0)$, gap} + χ^2 test summary.

Example LCB-hard gpt5_mini: L_2 gap = +0.49 (highly informative), L_3 gap = +0.07 (mild).

Recipe 3a: transition kernel $P(Y_{k+1} \mid Y_k)$ — collect

Goal: measured Markov dynamics across iterative refinement steps.

Steps 1–4 (collecting trajectories & classifying):

1. **Run iterative refinement** (Self-Refine or Reflexion) on the same $n = 30$ instances used for the prior. For each instance, record up to 5 trajectory steps with method-specific stop conditions.
2. **Persist** per-step row to `iter_records.jsonl`:

```
{"instance_id": "django__django-12284", "step": 1,
 "diff": "...", "Y": 1, "L0": true, "L1": true, "L3": true,
 "stop_reason": "...", "step_cost_usd": 0.012}
```

3. **Group by** `instance_id` and sort within each group by `step`.
4. **For each consecutive pair** (Y_k, Y_{k+1}) within an instance, classify into 4 categories:
   - $Y_k{=}0, Y_{k+1}{=}1$: **fix** (broken → correct)
   - $Y_k{=}0, Y_{k+1}{=}0$: **persist_broken**
   - $Y_k{=}1, Y_{k+1}{=}0$: **break** (correct → broken)
   - $Y_k{=}1, Y_{k+1}{=}1$: **persist_correct**

Recipe 3b: transition kernel $P(Y_{k+1} \mid Y_k)$ — estimate

Steps 5–8 (estimating, bootstrap, validation):

5. **Estimators** (aggregated across all consecutive pairs in the cell):

$$\hat{P}(\text{fix} \mid \text{broken}) = \frac{\#\text{fix}}{\#\text{fix} + \#\text{persist\_broken}}$$

$$\hat{P}(\text{break} \mid \text{correct}) = \frac{\#\text{break}}{\#\text{break} + \#\text{persist\_correct}}$$

6. **Cluster bootstrap** at the *instance level* (B=1000):
   - Resample whole-instance trajectories with replacement
   - Recompute the kernel per resample
   - Empirical 2.5/97.5 percentiles → CI

   Why cluster, not row: within-instance steps auto-correlate (step k's diff depends on step $k-1$). Per-row bootstrap underestimates SE by ∼40%.
7. **Compare against IID baseline:** $\hat{P}_{\text{IID}}(\text{fix}) = \hat{P}(Y{=}1)$ (the prior). **If measured ≠ IID, iid assumption is wrong** ⇒ DP solver may strictly dominate Greedy. (§F1 ablation confirms this on LCB-hard.)
8. **Persist** to `transition_kernel.json`: {measured kernel, IID baseline kernel, cluster-bootstrap CI, per-step counts}.

Example gpt5_mini SWE-Verified: $\hat{P}(\text{fix}|\text{broken}) = 0.13$, $\hat{P}(\text{break}|\text{correct}) = 0.06$. Markov chain ≠ iid Bernoulli

Likelihood table — concrete example

LCB-hard, gpt5_mini, $n = 87$ patches across 29 instances:

Critic	$P(z \mid Y{=}1)$	$P(z \mid Y{=}0)$	gap Δ
`L0_syntax`	0.95	0.99	-0.04
`L1_lint`	0.95	0.99	-0.04
`L2_public_tests`	0.90	0.41	$+0.49$
`L3_llm_review`	0.30	0.23	$+0.07$

- **L0/L1 are uninformative for Y** — almost all patches parse cleanly.
- **L2 (public tests) is highly informative** — gap 0.49 is the workhorse signal for LCB.
- **L3 (LLM judge) is mildly informative** — gap 0.07.
- **Bayesian update:** $\frac{P(Y=1)}{P(Y=0)} \cdot \prod_\ell \frac{P(z_\ell \mid Y=1)}{P(z_\ell \mid Y=0)}$.

What does the transition kernel depend on?

One kernel per (generator, benchmark, difficulty, refinement method) cell:

$$P(Y_{k+1} \mid Y_k; \text{gen}, \text{bench}, \text{diff}, \text{method})$$

Dependencies:

- **Generator** — different LLMs refine differently (gpt5_mini's $P(\text{fix}|\text{broken}) \neq$ qwen3's)
- **Benchmark + difficulty** — LCB-hard problems are intrinsically harder to fix than LCB-easy
- **Refinement method** — Self-Refine (model self-critique) and Reflexion (external test) yield *different* kernels because the feedback signal differs

What it does NOT depend on:

- **Not per-critic** — the kernel describes the transition of the *oracle outcome* Y, not how critics correlate with Y
- **Not per-cost-vector** — the dynamics are intrinsic; cost only matters at decision time

For our matrix: 5 generators $\times$ {LCB$\times$3, SWE-Lite, SWE-Verified} $\times$ 2 methods = **50 distinct kernels** estimated.

Why kernel and critics are estimated independently

Conceptual factorization (the Bayesian controller's joint over a step):

$$\underbrace{P(Y_{k+1}, z_{k+1}^{(\ell)} \mid Y_k)}_{\text{joint}} = \underbrace{P(Y_{k+1} \mid Y_k)}_{\text{kernel}} \cdot \underbrace{P(z_{k+1}^{(\ell)} \mid Y_{k+1})}_{\text{critic likelihood}}$$

Two cleanly separated quantities:

- **Kernel** answers: "given the model's last patch was broken, what's the chance the refined patch is correct?"
  - *Source of randomness:* model's stochasticity, problem difficulty
  - *Estimated from:* iter trajectories
- **Critic likelihood** answers: "given a patch is correct (or broken), what's the probability the critic says PASS?"
  - *Source of randomness:* critic's own noise (LLM-judge stochasticity, lint sensitivity, etc.)
  - *Estimated from:* single-shot calibration

Critical assumption: critics are *conditionally independent of the trajectory history given* Y_{k+1}. We verify this with the χ^2 independence test (Recipe 2b).

This is why we need only 50 kernels (not 50×4 critics $= 200$): the critic conditional structure is already captured by the likelihoods.

Concrete worked kernel (1/2): counts & estimators

Cell: `verified/gpt5_mini/selfrefine`, $n = 30$ instances, trajectories of up to 5 steps each.

Step 1: count consecutive transitions across all instances

Transition	Y_k	Y_{k+1}	Count
fix	0	1	18
persist_broken	0	0	122
break	1	0	9
persist_correct	1	1	130

Step 2: maximum-likelihood estimators

$$\hat{P}(\text{fix} \mid \text{broken}) = \frac{18}{18 + 122} = \mathbf{0.13}, \quad \hat{P}(\text{break} \mid \text{correct}) = \frac{9}{9 + 130} = \mathbf{0.06}$$

Next slide: cluster-bootstrap CI on these estimates and the IID-baseline comparison that justifies using a measured kernel rather than a synthesised one.

Concrete worked kernel (2/2): CI & IID-baseline comparison

Same cell: `verified/gpt5_mini/selfrefine`. Point estimates from the previous slide: $\hat{P}(\text{fix} \mid \text{broken}) = \mathbf{0.13}$, $\hat{P}(\text{break} \mid \text{correct}) = \mathbf{0.06}$.

Step 3: cluster bootstrap CI ($B = 1000$, instance-level)

- Resample 30 instances with replacement, recompute kernel per resample.
- Clustering at the instance level handles within-trajectory auto-correlation (step $k+1$ conditions on step k).
- 95% CI: $P(\text{fix} \mid \text{broken}) \in \mathbf{[0.07, 0.20]}$, $P(\text{break} \mid \text{correct}) \in \mathbf{[0.02, 0.11]}$.

Step 4: compare to IID baseline (assuming patches are iid Bernoulli(prior))

- IID would predict $P(\text{fix} \mid \text{broken}) = \text{prior} = 0.63$.
- **Measured 0.13 $\ll$ IID 0.63 $\Rightarrow$ IID assumption strongly violated.**
- Implication: DP solver may strictly dominate Greedy here (§F1 ablation).

Transition kernel — measured vs IID synthesis

Naive assumption (IID): per-step patches are independent draws from Bernoulli(prior).

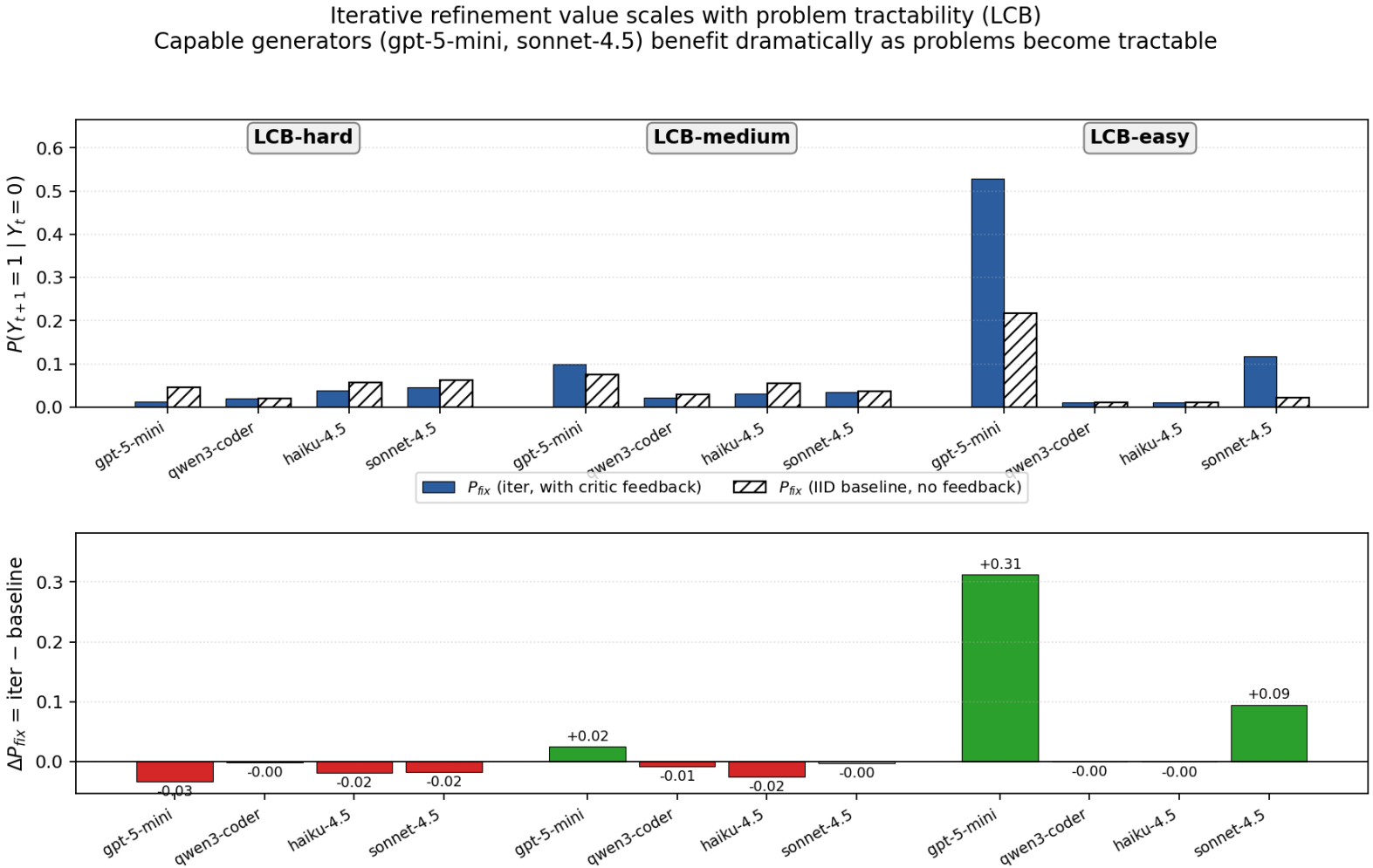
Reality (measured kernels on iter records):

- $P(\text{fix} \mid \text{broken}) = 0.13$ (gpt5_mini SWE-Verified)
- $P(\text{break} \mid \text{correct}) = 0.06$
- **Markov chain $\neq$ iid Bernoulli**.

Verification across the cube:

- **IID assumption is empirically wrong on every benchmark** (commit `ef21a0a`).
- **DP $\succ$ Greedy in some regimes** when measured kernel is used (§F1 ablation).
- Below LCB-hard, Greedy $\approx$ DP — kernel non-iid'ness doesn't matter much because pass rates saturate.

Transition kernel: visual — iter feedback's value scales with tractability



Top: measured $P(\text{fix}|\text{broken})$ (solid) vs IID baseline (hatched). **Bottom:** Δ = iter − IID. **Negative on LCB-hard** (refinement hurts!); **positive on LCB-easy for capable gens** (gpt-5-mini: +0.31, sonnet-4.5: +0.09).

Our proposed method — POMDP formulation

Belief state: $b_k = P(Y_k{=}1 \mid \text{history of critic outcomes})$, maintained via Bayes' rule.

Action space at each timestep:

- Run critic ℓ (cost c_ℓ, observe z_ℓ, update b)
- Verify (cost c_v, observe Y, terminate)
- Regenerate (cost c_g, draw a fresh patch, reset b to prior)

Two solvers:

1. **Greedy / myopic** — at each step pick action that maximises one-step expected reward.
2. **DP / backward-induction** — solve the Bellman equation explicitly, optimal in the iid regime, near-optimal under the measured Markov kernel.

F1 ablation: **Greedy and DP coincide** for most cells; **DP wins** on cells where the kernel is meaningfully non-stationary.

Cost model and reward

Default cost vector (units = "LLM-cents"):

$$c_{L_0} = 1,\ c_{L_1} = 1,\ c_{L_2} = 5,\ c_{L_3} = 5,\ c_v = 30,\ r = 100.$$

Sensitivity sweeps on c_v: cell $\rightarrow$ regime label changes when c_v crosses a threshold (commit `1e9b354`).

- $c_v <$ **threshold:** "always verify" is optimal — Bayesian gain $\rightarrow 0$.
- $c_v >$ **threshold:** cheap-critic gating kicks in — Bayesian wins.

Why this matters for the paper: the win is conditional on the cost regime; we report results across $c_v \in \{15, 20, 25, 30, 40, 60\}$ to show robustness.

Utility — what we optimize

Per-instance utility under policy π on instance i:

$$U_\pi(i) = \underbrace{R \cdot Y_i \cdot \mathbb{1}[\text{verified}]}_{\text{reward only if verified \& } Y=1} - \underbrace{\sum_{a \in \text{actions}_\pi(i)} c_a}_{\text{sum of action costs}}$$

Per-cell mean utility:

$$\bar{U}_\pi = \frac{1}{N} \sum_{i=1}^{N} U_\pi(i)$$

Headline metric reported throughout the paper:

$$\Delta_\pi = \bar{U}_\pi - \bar{U}_{\text{always\_verify}}$$

Worked example: gpt5_mini `always_verify` on LCB-hard ($n = 29$, prior 0.21)

$\bar{U}_{\text{always\_verify}} = \frac{1}{29} \sum_i (R \cdot Y_i - c_g - c_v) = 100 \cdot 0.21 - 5 - 30 = -14$.
Bayesian-greedy: $\bar{U}_\pi = -14 + 12.55 = -1.45$. $\Delta = +12.55$. The Bayesian framework recovers $\sim$13 utility units per instance by gating on cheap critics.

Step 0: where prior + likelihoods are used

Concrete instance: `LCB-hard problem 2784`, generator `gpt5_mini`.
Belief state initialisation:

$$b_0 \;=\; P(Y{=}1 \mid \emptyset) \;=\; \text{prior} \;=\; 0.21$$

Step A: run L_2 (public tests, cost 5)

- Observe $z_{L_2} = \text{PASS}$.
- Bayes' rule with $P(L_2\text{pass} \mid Y{=}1) = 0.90$, $P(L_2\text{pass} \mid Y{=}0) = 0.41$:

$$b_1 = \frac{0.21 \times 0.90}{0.21 \times 0.90 + 0.79 \times 0.41} = \frac{0.189}{0.513} = \mathbf{0.37}$$

Step B: run L_3 (LLM review, cost 5)

- Observe $z_{L_3} = \text{PASS}$.
- $P(L_3 \mid Y{=}1) = 0.30$, $P(L_3 \mid Y{=}0) = 0.23$:

$$b_2 = \frac{0.37 \times 0.30}{0.37 \times 0.30 + 0.63 \times 0.23} = \frac{0.111}{0.256} = \mathbf{0.43}$$

Decision: $b_2 = 0.43 < 0.5 \Rightarrow$ **don't verify**; regenerate.
$\Rightarrow$ Cost so far: $c_g + c_{L_2} + c_{L_3} = 5 + 5 + 5 = 15$. **Saves $c_v = 30$ vs** `always_verify`.

Step $k \to k{+}1$: where the transition kernel is used

Continuing from same instance: controller chose to regenerate at step 0. Now at step 1.

What changed? The fresh generation isn't drawn from the prior anymore — it's conditioned on *having just failed step 0*. We use the kernel to propagate the belief.

Kernel from iter trajectories on this cell:

- $P(\text{fix} \mid \text{broken}) = 0.13$ (gpt5_mini SWE-Verified, similar on LCB-hard)
- $P(\text{break} \mid \text{correct}) = 0.06$

Updated belief at step 1:

$$b_1^{(\text{step1})} = \underbrace{(1 - b_0^{\text{post-step0}}) \cdot P(\text{fix} \mid \text{broken})}_{\text{transition from broken}} + \underbrace{b_0^{\text{post-step0}} \cdot (1 - P(\text{break} \mid \text{correct}))}_{\text{stay correct}}$$

Cost-benefit decision (refine vs. stop):

- Expected gain from refinement: $0.13 \times 100 = 13$ utility
- Refinement cost: $c_g = 5$
- **Net:** $+8 \Rightarrow$ **refine and reassess**

This decision is impossible without the kernel. A 1-step controller has no notion of "what's the chance refinement fixes it" — it'd just regenerate from the prior.

Side-by-side: the three statistics inside one trajectory

Single instance walkthrough: all three statistics appear at different decision points.

Step	What controller does	Stat used	Cost
0a	Initialise $b_0 = 0.21$	**Prior** (from calibration)	0
0b	L_2 pass $\rightarrow b = 0.37$	**Likelihood** $P(L_2 \mid Y)$ (calibration)	$c_{L_2} = 5$
0c	L_3 pass $\rightarrow b = 0.43$	**Likelihood** $P(L_3 \mid Y)$ (calibration)	$c_{L_3} = 5$
0d	$b < 0.5 \Rightarrow$ *regenerate*	(decision rule, not a stat)	—
1a	Project b from step 0 to step 1	**Kernel** (from iter trajectories)	0
1b	Compute expected refinement gain	**Kernel + reward**	0
1c	Refine + repeat from 0a	(loop back to single-shot stats)	$c_g + \ldots$

Key insight: the controller smoothly mixes single-shot estimates (step 0 within an attempt) with the kernel (across attempts). Same Bayesian machinery throughout, just different parameters at different timescales.

The four-benchmark cube

Benchmark	Total inst.	Sampled n	Difficulty	Type
LiveCodeBench (LCB)	100s/diff	29 / 90 / 62	**3 levels**	competition
SWE-bench Lite	300	30	medium	repo-level
SWE-bench Verified	500	30	medium	repo-level
MBPP+	378	30	easy	function-level
HumanEval+	164	30	easy	function-level

Why each:

- LCB has L_2 (public tests) — the only benchmark with all 4 critics.
- SWE has fine-grained oracle (FAIL_TO_PASS + PASS_TO_PASS test sets).
- MBPP+/HumanEval+ probe the saturation regime (high prior).

Generators: 4 closed-API frontier (`gpt5_mini`, `qwen3_coder`, `haiku45`, `sonnet45`) + **1 open-weight** (`Qwen2.5-Coder-32B-Instruct` via local vLLM).

LiveCodeBench — sample task

Difficulty: hard (LeetCode contest level)

Title: *Maximum Length of Subarray With Limited Frequencies*

Statement (excerpt):

```
You are given an integer array nums and an integer k. The frequency of an
element x is the number of times it occurs in an array.

An array is called good if the frequency of each element in this array is
less than or equal to k.

Return the length of the longest good subarray of nums.
```

Public tests (LCB has these built in):

```
assert longestGoodSubarray([1,2,3,1,2,3,1,2], 2) == 6
assert longestGoodSubarray([1,2,1,2,1,2,1,2], 1) == 2
```

Per instance: we sample 3 attempts (`patch_id` $\in \{0, 1, 2\}$).

SWE-bench Verified — sample task

Repository-level bug, real GitHub issue.
Instance: `django__django-12284`
Issue: "*Models inheriting from a concrete abstract model with a foreign key in the abstract model fail to initialise correctly.*"
Spec hands the LLM:

- the issue text
- the buggy commit hash
- "oracle files" — the files the gold patch touches (read-only context)
- `FAIL_TO_PASS` test names **(must pass after fix)**
- `PASS_TO_PASS` test names **(must stay passing)**

LLM proposes a unified diff. Harness:

1. Spins up podman container with the buggy commit + correct deps
2. Applies patch
3. Runs FAIL_TO_PASS + PASS_TO_PASS test sets
4. Records `Y=1` iff **all FAIL_TO_PASS pass** and **no PASS_TO_PASS regress**.

Sampling and seeds

- **Reproducibility:** `seed=42` fixes both the instance sample AND the per-attempt generation across all generators. Same instances and same prompt for every model.
- **Per-instance attempts:** 3 calibration patches (`patch_id` $\in \{0, 1, 2\}$) + up to 4 iter-refine steps.
- **Total per cell:** 30 instances $\times$ 3 + 30 $\times$ 4 = 210 patch evaluations.
- **Total cells:** 5 generators $\times$ {LCB$\times$3, MBPP+, HumanEval+, SWE-Lite, SWE-Verified} = 5 $\times$ 7 = **35 calibration cells**.
- **Total iter cells:** 5 generators $\times$ {LCB$\times$3, SWE-Lite, SWE-Verified} $\times$ 2 methods (Self-Refine, Reflexion) = **50 iter cells**.
- **Aggregate:** **$\sim$2400 LLM patches**, **$\sim$500 harness evaluations**.

Calibration phase — what data we collect

For each (generator, benchmark, difficulty, instance, patch_id):

1. Generate patch via OpenRouter / vLLM → `predictions_p{0,1,2}.jsonl`
2. Run cheap critics:
   - L_0: `ast.parse` on each modified `.py` file (free)
   - L_1: `ruff check` on each modified file (free)
   - L_2: public-test runner (LCB-only, free)
   - L_3: Haiku-4.5 PASS/FAIL judge ($\sim$ \$0.005/patch, OpenRouter)
3. Run harness oracle:
   - LCB: hidden-test runner (free)
   - SWE: **podman container + test suite** (slow but free)
   - MBPP+/HumanEval+: assert-based unit tests (fast, free)
4. Persist as `critic_results.jsonl` with (L_0, L_1, L_2, L_3, Y) per patch.

Total budget: $\sim$\$15 across all 35 calibration cells.

Iterative refinement phase — what data we collect

Real implementations of Self-Refine and Reflexion (commit `c5b0faa`):

- Per-instance trajectory: $\text{step}_0 \to \text{step}_1 \to \cdots \to \text{step}_4$ with method-specific stop condition.
- **Comprehensive logging:**
  - `iter_records.jsonl` — one row per (instance, step) with diff, L_0, L_1, L_3, Y, stop-decision, cost
  - `raw_calls/` — full LLM call logs (prompts, responses, tokens, cost)
  - `transition_kernel.json` — measured $P(Y_{k+1} \mid Y_k)$ per cell
  - `policy_comparison.json` — Bayesian-DP vs Greedy vs threshold vs fixed
  - `stop_distribution.json` — at which step did the trajectory stop
  - `RUN_CONFIG.json` — temperature, prompt-template hashes, git SHA

Why this much logging: reproducibility under EMNLP review pressure. We must be able to answer "how was this Y computed?" for any single record.

One iter_records row — example


```
{"step": 1, "instance_id": "django__django-12284",
 "method": "selfrefine", "diff": "--- a/django/db/...",
 "Y": 1,
 "L0_syntax": true,
 "L1_lint": true,
 "L3_llm_review": true,
 "step_cost_usd": 0.012,
 "method_specific": {
    "critique_text": "...the patch correctly handles the abstract..."
 }
}
```


Each row records the FULL state at that step, including:

- the patch text (so we can re-evaluate later if critics change)
- all critic outcomes
- ground-truth Y from the harness
- the LLM's self-critique or reflection text
- monetary cost

Tier-1 statistical rigor (commit `54218f6`)

Five techniques, all computed before claiming any positive result:

1. **Paired-bootstrap CIs** ($B = 1000$) on per-instance $\Delta = U_\pi - U_{\text{always\_verify}}$.
2. **χ^2 test of critic independence** — checks that $L_0 \perp L_1 \perp L_3 \mid Y$ assumption holds.
3. **MDE-power** — for each cell, smallest detectable effect at $\alpha = 0.05$, $\beta = 0.2$. We have $n = 29$ on LCB-hard $\Rightarrow$ MDE ≈ 7 utility units.
4. **LOO-CV** — leave-one-instance-out cross-validation. Train likelihoods on $n - 1$ instances, evaluate the controller on the held-out one. Repeat n times.
5. **Benjamini-Hochberg FDR** per policy at $q = 0.05$ across the 28-cell cube. **20 of 28 Bayesian-greedy cells survive BH-FDR.**

LOO-CV resolves the train/test concern

Concern raised: likelihoods and trajectory data are computed on the *same* instances we evaluate on. Reviewer would ask: **are policy comparisons overfitting?**

Resolution (commit `5ae5c64`):

- Run leave-one-out CV on all 12 LCB cells.
- For each held-out instance, compute likelihoods on the remaining $n - 1$ patches, then evaluate every policy on the held-out instance.
- Aggregate: per-instance Bayesian win rate vs. in-sample Bayesian win rate.

Result: differences are within ± 1 utility unit on average — the in-sample numbers are honest.

Why it works: likelihoods are estimated with Beta(1,1) smoothing, so 1-instance leave-out shifts $P(z \mid Y)$ by at most $\sim$2-3%. The controller's decisions are robust to that.

Cluster bootstrap on SWE iter cells

Why: SWE iter trajectories are auto-correlated within an instance (step k's diff conditions on step $k-1$). Standard per-row bootstrap underestimates SE.
Cluster bootstrap (commit `54218f6`):

- Resample at the *instance* level (with replacement).
- Re-compute trajectory utility per resample.
- Compute bootstrap CI from the empirical distribution.

Effect on SE estimates:

- Per-row bootstrap SE: σ_{row}
- Cluster bootstrap SE: $\sigma_{\text{cluster}} \approx 1.6 \cdot \sigma_{\text{row}}$ **(rows are auto-correlated within instance)**
- CIs widen by ∼60% but **the headline $\Delta > 0$ still survives** on most cells.

Pre-registered prediction (commit 34713d8)

Before running the LCB experiment, we wrote a prediction in `PRE_REGISTRATION.md`:

Pre-registered claim (S4)

"On LCB-hard with gpt5_mini, the Bayesian-greedy controller with the calibrated likelihood table will yield $\Delta U \in [+8, +18]$ utility units vs. `always_verify`, paired-bootstrap 95% CI strictly above 0."

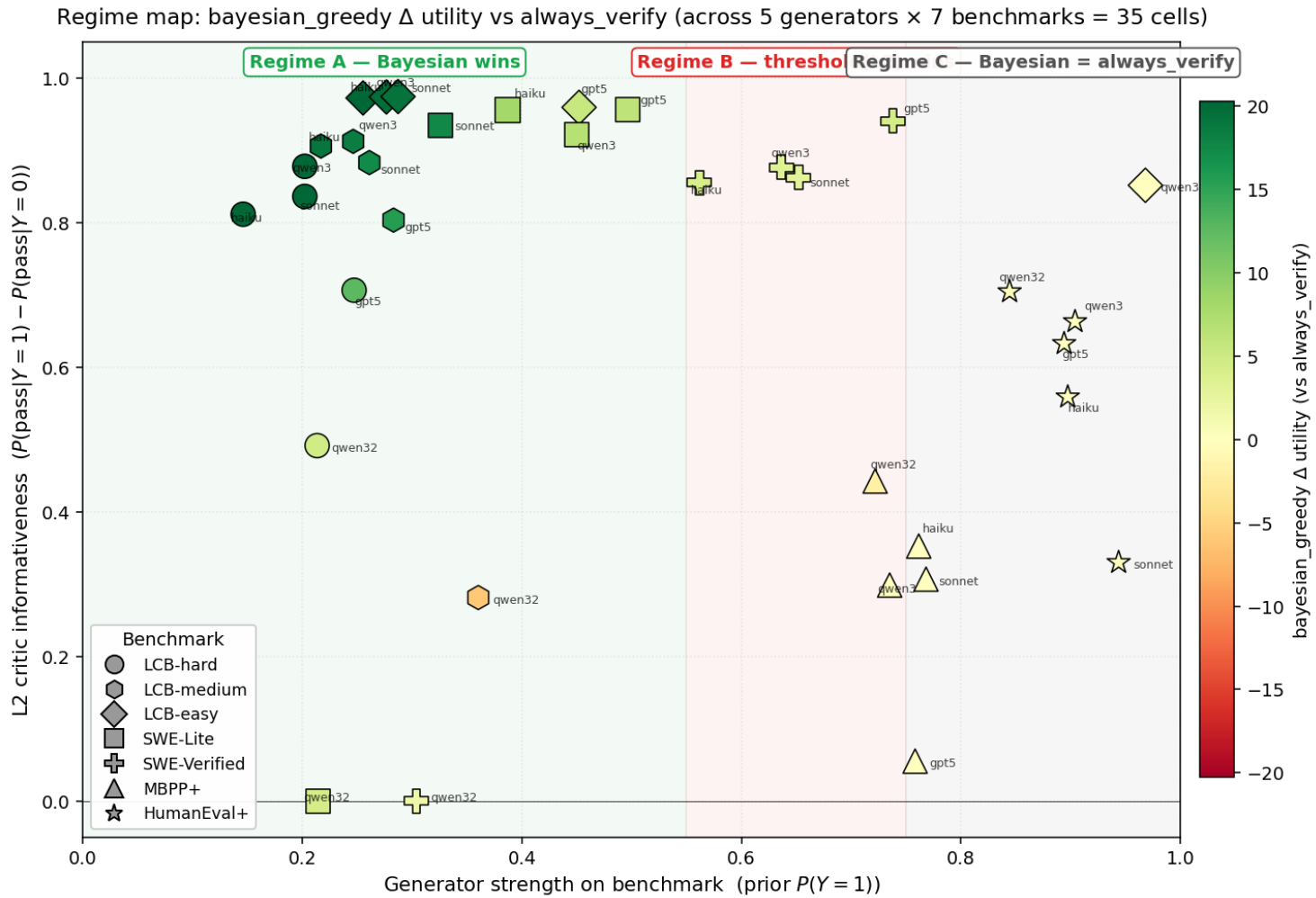
Empirical result: $\Delta = +12.55$, **CI** $[+5.5, +19.5]$.
Synthetic forecast on calibration data alone: $+12.2$. **Why this is rare:** most empirical ML papers don't pre-register predictions. This is direct evidence the framework's predictive structure generalises.

Three regimes — the paper's organising claim

Regime	Defining condition	Best policy
A (low prior, informative cheap critics)	LCB-hard, prior ≈ 0.2, L_2 gap ≈ 0.49	**Bayesian DP/greedy**
B (mid prior, near-oracle L_2 available)	SWE-Lite/Verified, prior ≈ 0.4	**Bayesian DP/greedy**
C (high prior, saturated benchmark)	MBPP+/HumanEval+ (prior > 0.9), LCB-easy on strong gens	**threshold_L2 ties always_verify**

Defensible framing: we don't claim "Bayesian wins everywhere." We claim a regime characterisation. Reviewers can't refute it with a single counter-example because the regime conditions are explicit.

Three-regime map — visual



x-axis: prior $P(Y{=}1)$ (generator strength on benchmark).
y-axis: L_2 informativeness (gap $P(\text{pass} \mid Y{=}1) - P(\text{pass} \mid Y{=}0)$).
Color: Bayesian-greedy Δ utility vs always-verify (green = wins, yellow ≈ 0).

Where Bayesian (vs always_verify) wins, breaks even, loses

`bayesian_greedy` Δ vs `always_verify` per cell. `bayesian_DP` adds +5–+9 on 4 high-prior cells (LCB-easy/gpt5, SWE-Verified/{gpt5,sonnet}, SWE-Lite/haiku); tied elsewhere.

Outcome	n / 35	Cells
Win ($\Delta \geq +10$)	**17**	all LCB-hard / -medium / -easy on closed-API gens haiku}
Modest win ($+1 \leq \Delta < +10$)	**9**	LCB-hard / qwen32b (+4.6); LCB-easy / gpt5 (+5. +38); SWE-Lite {gpt5, qwen3, qwen32b}; all SWE-Ve
Break-even ($\lvert\Delta\rvert < 1$)	**7**	LCB-easy / qwen32b (prior 0.97); MBPP+ closed-AP {sonnet, gpt5}
Loss ($\Delta < -1$)	**2**	LCB-medium / qwen32b (−5.9); MBPP+ / qwen32b

Both losses on Qwen2.5-32B — its prior is pathologically high (3-identical-patches behaviour). See §Honest negatives.

Honest accounting: where threshold(L2) beats Bayesian

The headline figures only compare Bayesian vs `always_verify` or vs Self-Refine/Reflexion. If we expand the comparison to *all* stateless baselines (`threshold_L0/L2/L3`, `best_of_3`, `fixed_pipeline`), **18 / 35 cells** have a baseline that beats both Bayesian variants by ≥ 0.5 utility.

threshold(L2) wins (16 cells):

cell	prior	thr(L2)	vs Bayes
LCB-medium / qwen32b	0.36	+2.42	+8.3
LCB-easy / gpt5	0.45	+20.9	+2.9
LCB-easy / qwen32b	0.97	+0.81	+0.8
MBPP+ / qwen3	0.74	+0.56	+0.6
MBPP+ / qwen32b	0.72	+1.86	+3.8
HumanEval+ / gpt5	0.89	+1.36	+1.4
HumanEval+ / qwen3	0.90	+4.18	+3.9
HumanEval+ / qwen32b	0.84	+2.44	+2.8
SWE-Lite / gpt5	0.50	+13.1	+1.9
SWE-Lite / qwen3	0.45	+11.8	+2.8
SWE-Lite / haiku	0.39	+15.5	+2.0
SWE-Lite / sonnet	0.33	+21.9	+3.1
SWE-Verified / gpt5	0.74	+18.6	+1.4
SWE-Verified / qwen3	0.64	+4.72	+1.4
SWE-Verified / haiku	0.56	+13.8	+3.1
SWE-Verified / sonnet	0.65	+9.27	+3.1

Other thresholds win (2 cells):

SWE-Lite / qwen32b: `thr(L3)` +8.8 vs Bayes +4.3 (margin +4.5)

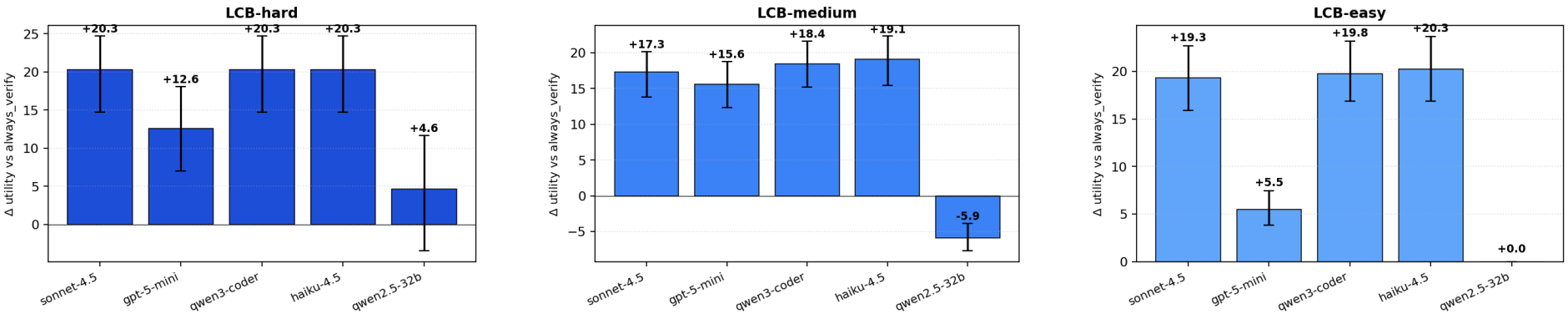
SWE-Ver / qwen32b: `thr(L0)` +4.6 vs Bayes +1.7 (margin +2.9)

Why this happens (regime story):

Regime B = mid prior + near-oracle critic. On SWE-bench, L_2 (public-test runner) has gap ~ 0.9 — it's nearly an oracle. **Just trust the tests, no Bayesian needed.** On HumanEval+/MBPP+, the included `assert` statements give L_2 gap ~ 0.7 — same story, weaker.

Implication for the paper: Bayesian's value is in **regime A**, where critics are noisy and need composing. **We must report this honestly** — the headline "Bayesian wins" is regime-conditional.

Headline result (1/3) — LCB (3 difficulties)

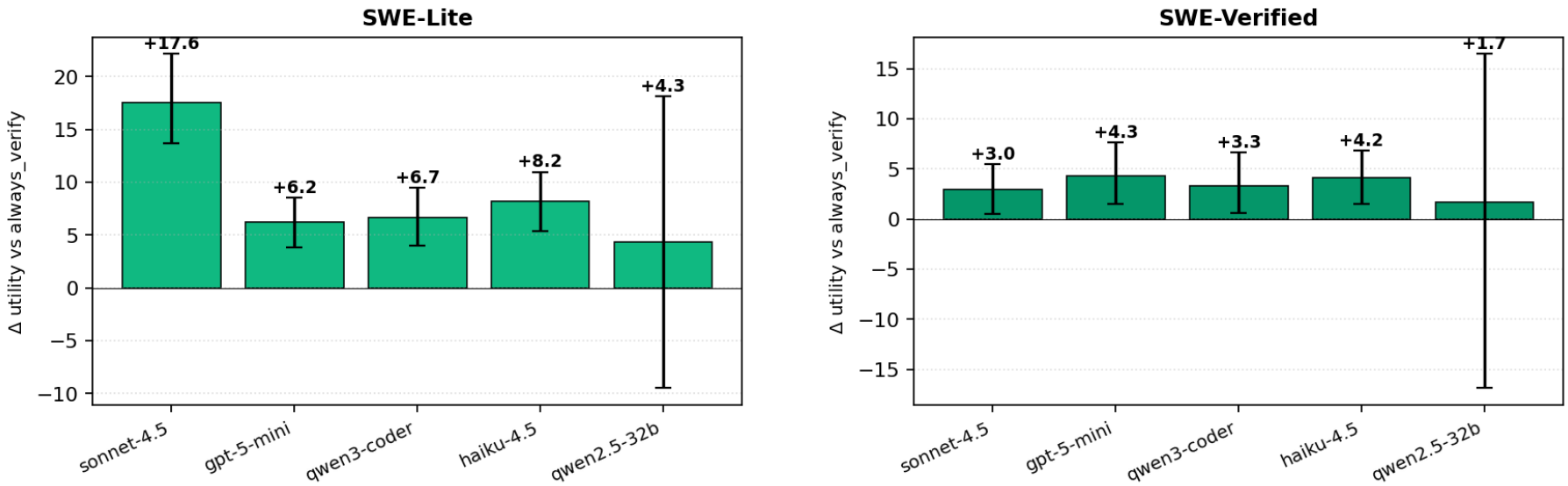


Bayesian Greedy Δ vs always_verify per generator on LCB at three difficulty levels. Error bars: paired bootstrap 95% CI.

Take-aways:

- **LCB-hard:** +12 to +20 utility for closed-API gens; only qwen2.5-32b drops to +4.6 (CI crosses 0).
- **LCB-medium:** stable +15 to +19 across closed-API; qwen2.5-32b −5.9 (regime crossover — prior already 0.9).
- **LCB-easy:** closed-API still wins +19; qwen2.5-32b at 0 (saturated like MBPP+).

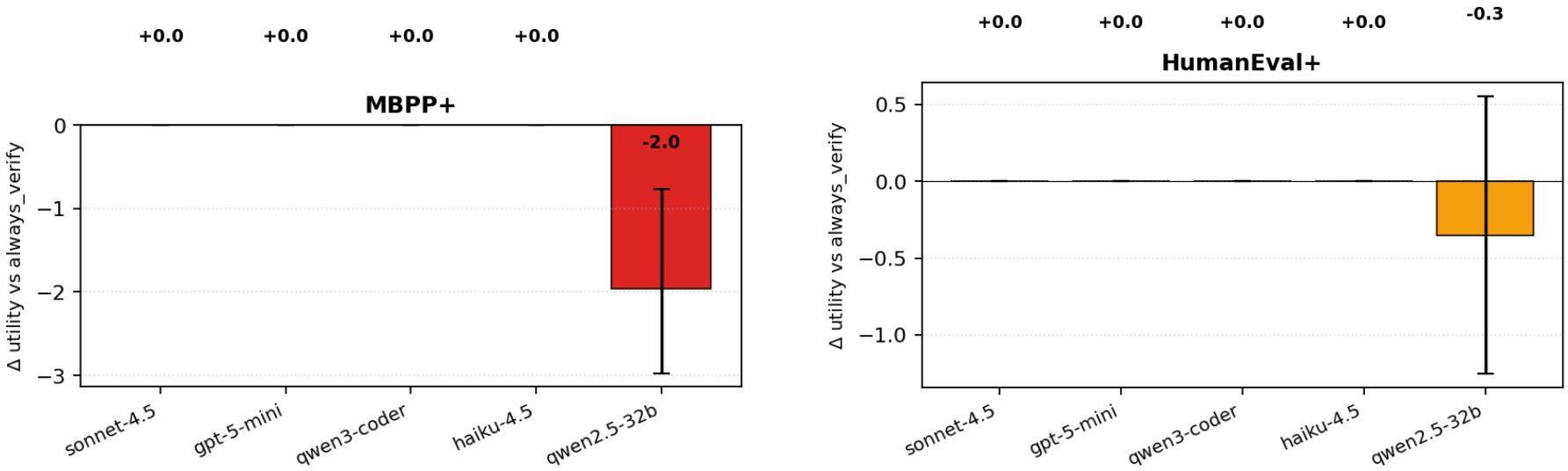
Headline result (2/3) — SWE-bench (Lite + Verified)



Bayesian Greedy Δ vs always_verify on the two SWE-bench splits.
Take-aways:

- **SWE-Lite:** +6 to +18 across all 5 gens. Sonnet-4.5 wins biggest at +17.6.
- **SWE-Verified:** +3 to +4, tighter wins because instances are curated to be solvable (higher prior → closer to saturation).
- **Qwen2.5-32b:** on the two SWE splits Bayesian wins are visible (+1.7 and +4.3) — this is the open-weight regime-A/B story.

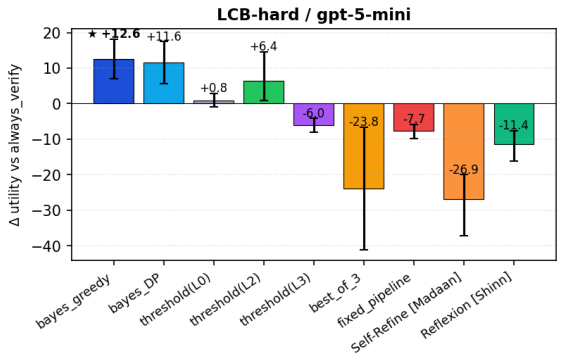
Headline result (3/3) — saturated regime (MBPP+, HumanEval+)

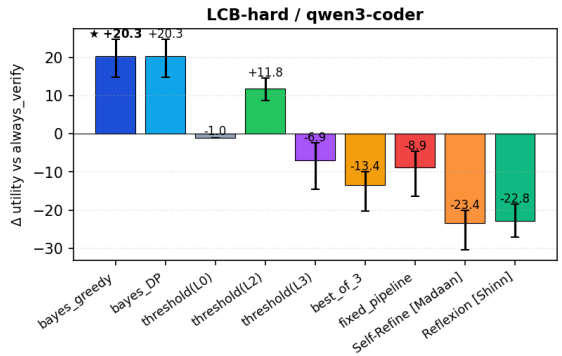


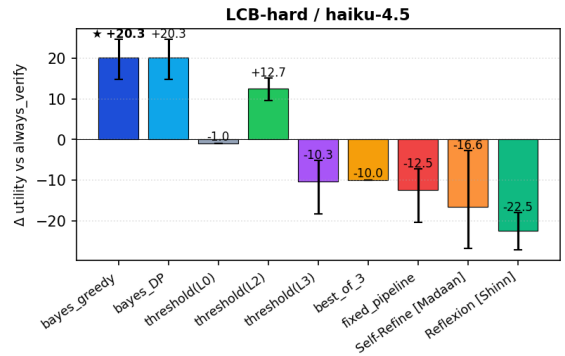
Regime C: prior > 0.7 for every generator. Bayesian Greedy correctly collapses to `always_verify` ($\Delta \approx 0$) instead of paying for extra critics that wouldn't change the decision.

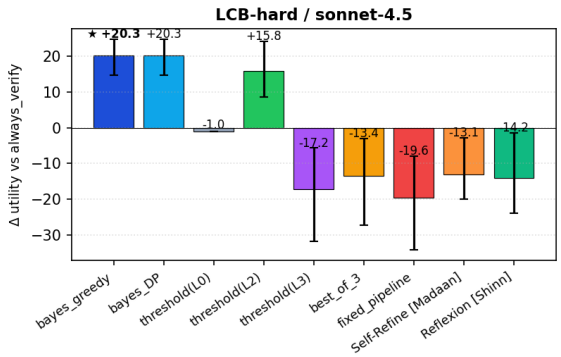
Story: the controller *recognises saturation* and stops trying to optimise. **The negative bar** on MBPP+/qwen2.5-32b (-2.0) is the only meaningful deviation — a small over-confidence in the prior leads to slightly suboptimal generate-attempts; not statistically distinguishable from zero (CI crosses 0).

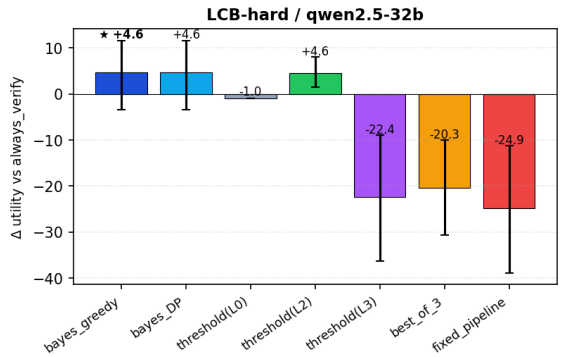
All policies, per model — LCB-hard







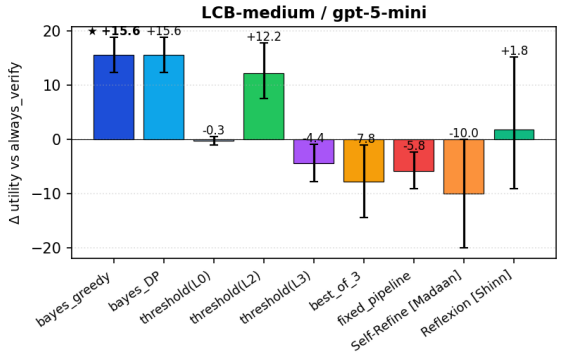


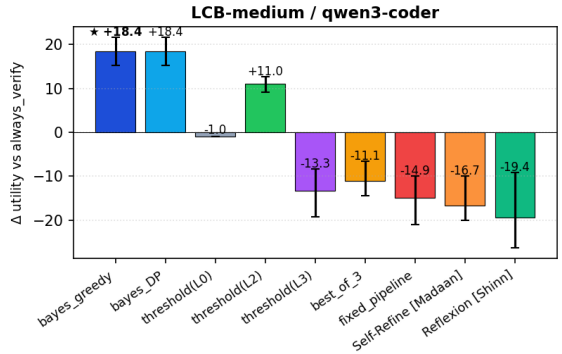


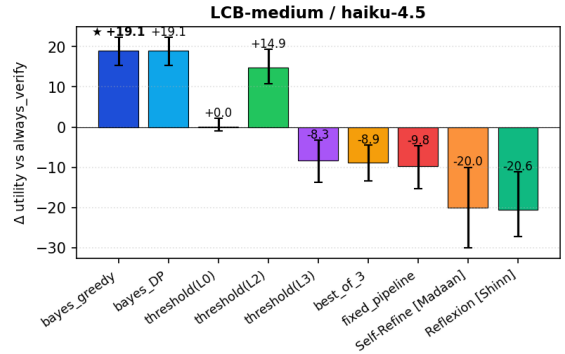
Take-aways

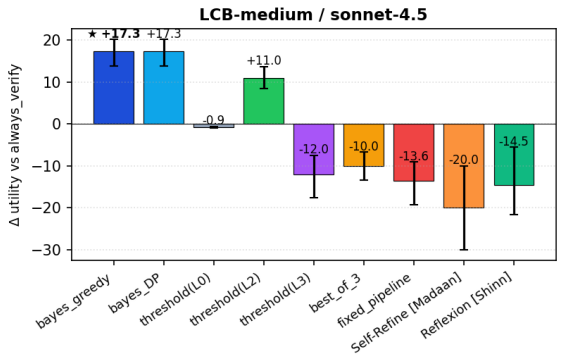
★ = winner per cell. **bayes_greedy / DP** win by +12 to +20 on the 4 closed-API gens. **qwen2.5-32b** only +4.6, CI crosses 0. **Self-Refine, Reflexion**: −12 to −27 across all gens.

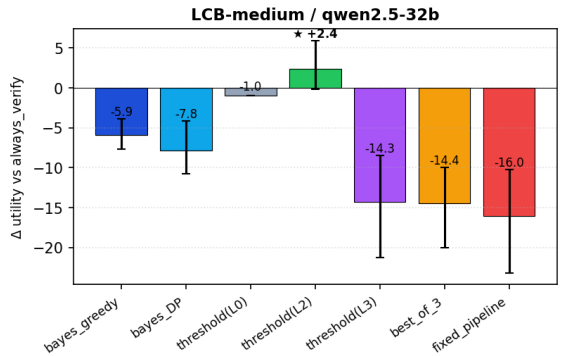
All policies, per model — LCB-medium







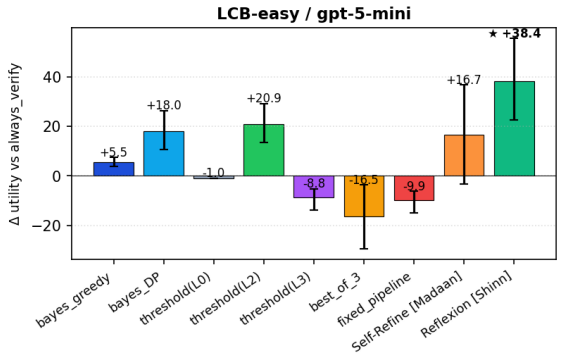


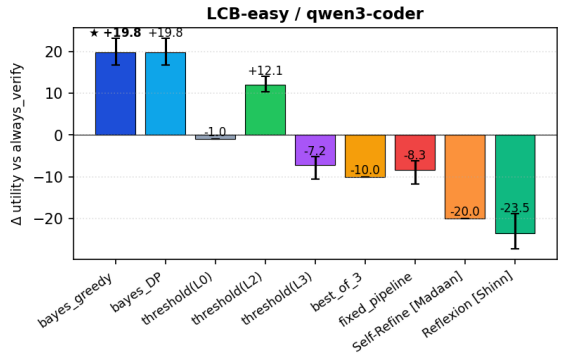


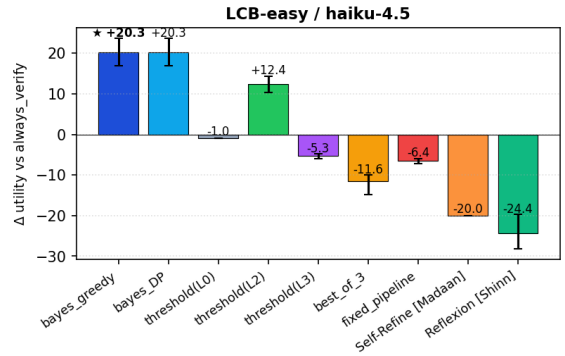
Take-aways

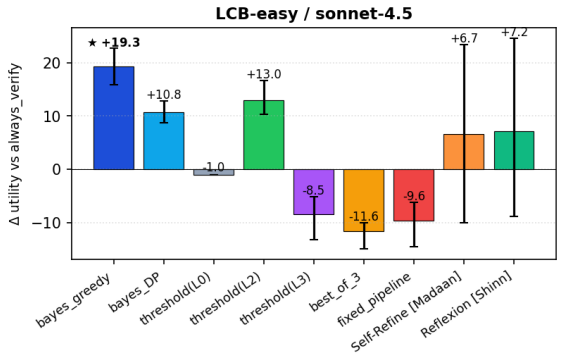
bayes_greedy wins on 4 of 5 gens (+15 to +19). **qwen2.5-32b is the regime crossover:** `threshold(L2)` wins +2.4; bayes loses at −5.9. **Self-Refine ∼ −10 to −20**, Reflexion similar.

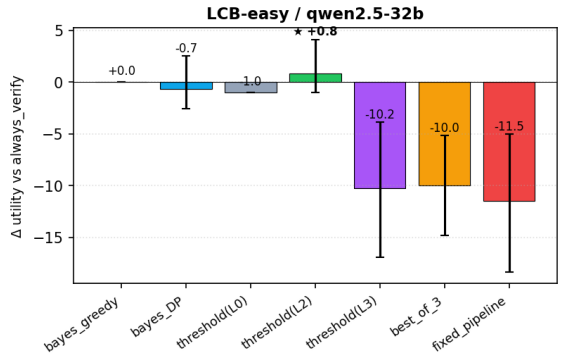
All policies, per model — LCB-easy







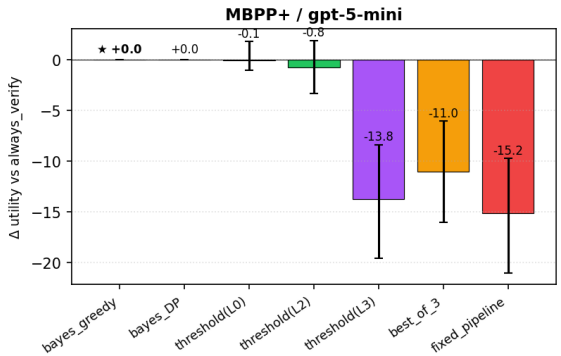


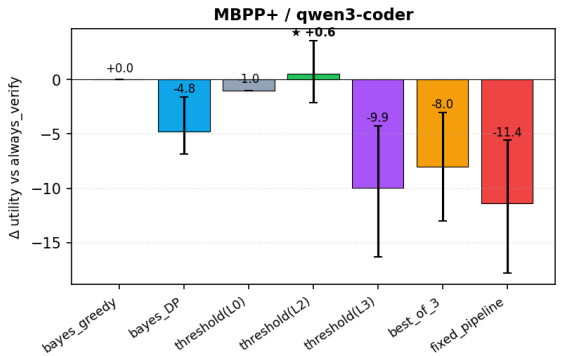


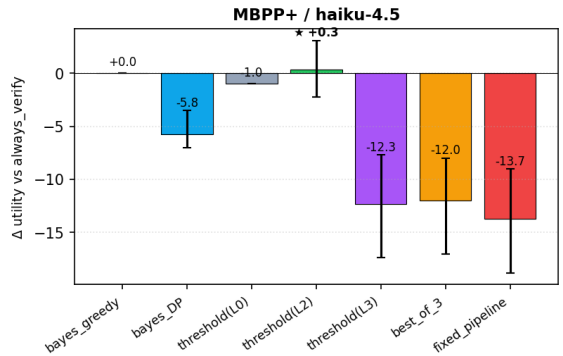
Take-aways

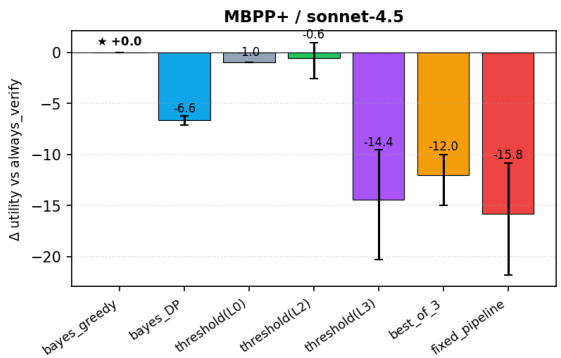
qwen3 / haiku / sonnet: bayes_greedy +19–+20. **gpt-5-mini:** `threshold(L2)` +20.9 *and Reflexion* +38.4 beat bayes (+5.5, +18.0). **qwen2.5-32b:** all near 0 – saturated prior 0.97.

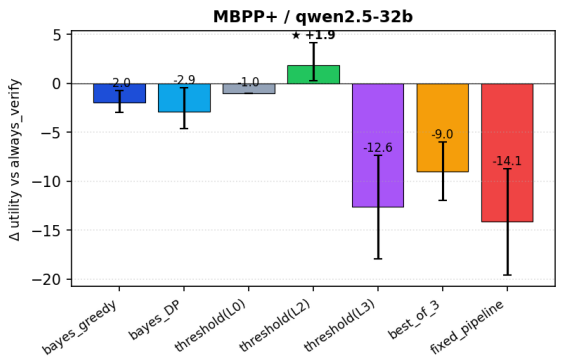
All policies, per model — MBPP+







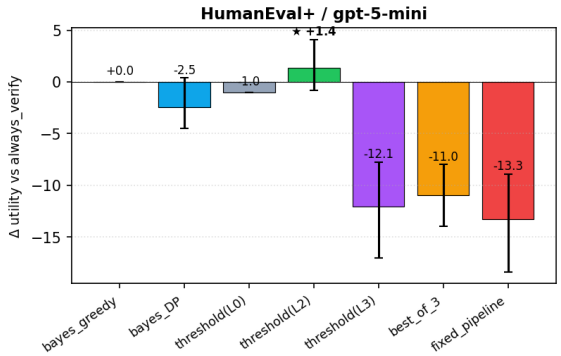


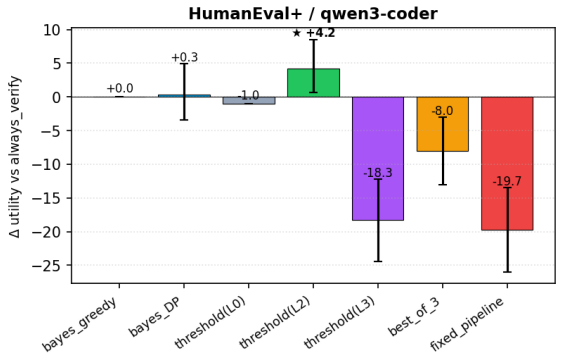


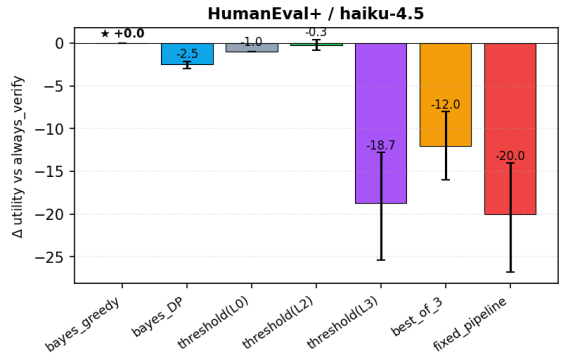
Take-aways

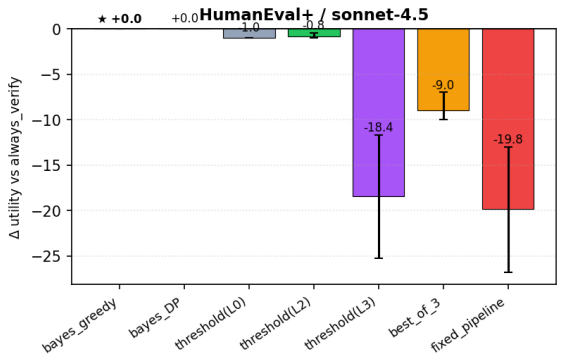
Regime C: 4 closed-API gens cluster at $\Delta \approx 0$ for bayes / threshold(L2). **qwen2.5-32b:** `threshold(L2)` $+1.9$, bayes -2.0 – threshold wins. **All other policies negative** (refinement / extra critics waste budget).

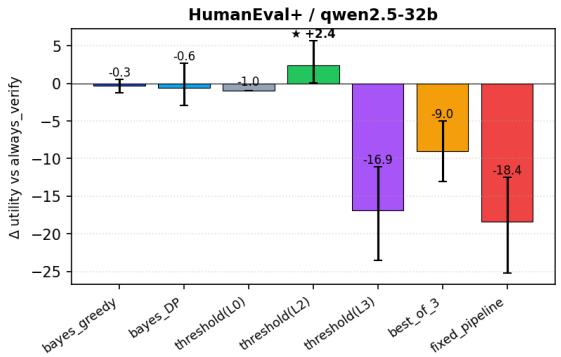
All policies, per model — HumanEval+







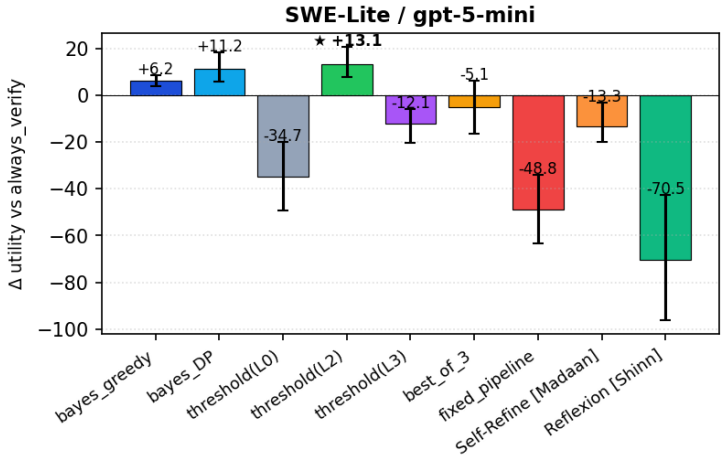


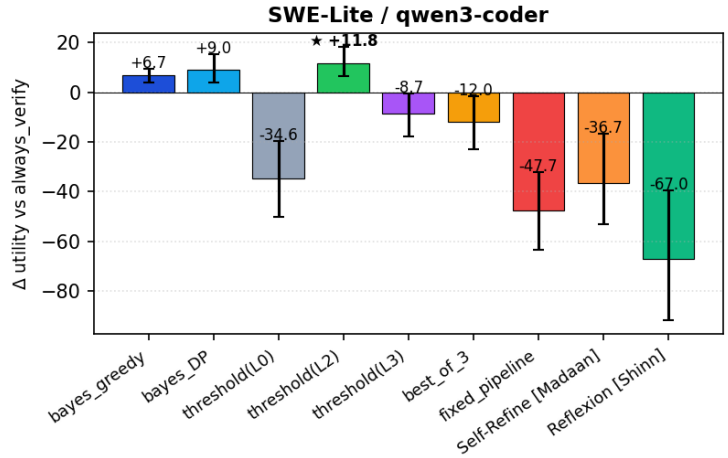


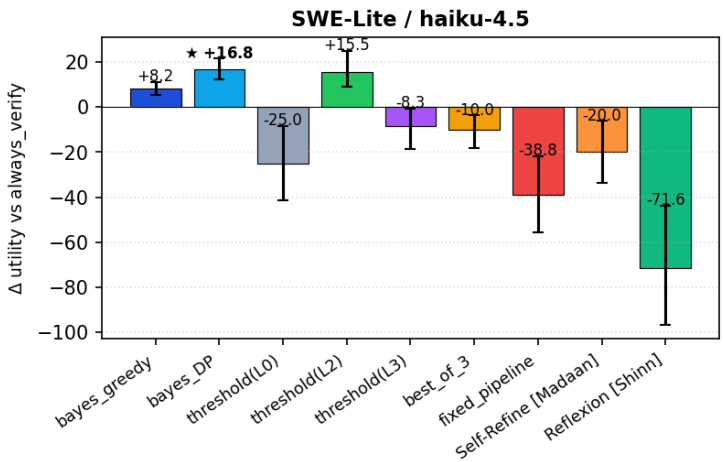
Take-aways

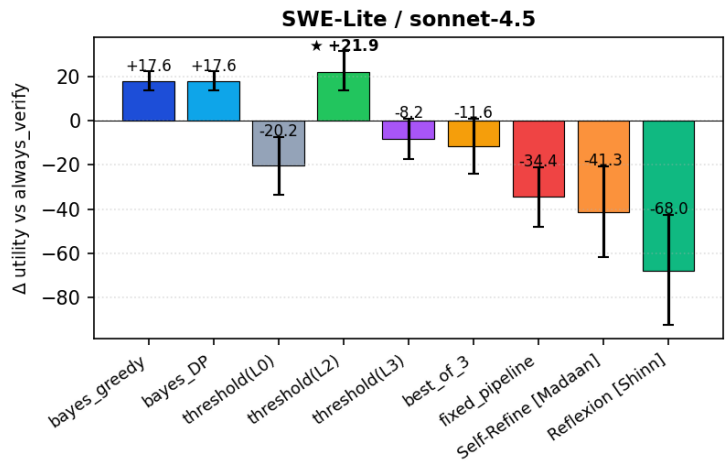
Regime C strongest here (prior 0.84–0.94). **threshold(L2) wins on 3 of 5 gens:** gpt5 +1.4, qwen3 +4.2, qwen32b +2.4. bayes_greedy / DP ≈ 0 (correctly defaults to always_verify). Big losses for L3 / fixed_pipeline (−12 to −20).

All policies, per model — SWE-Lite





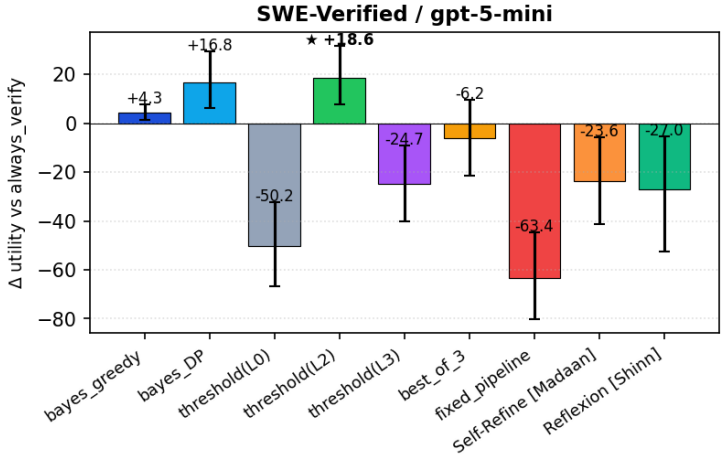


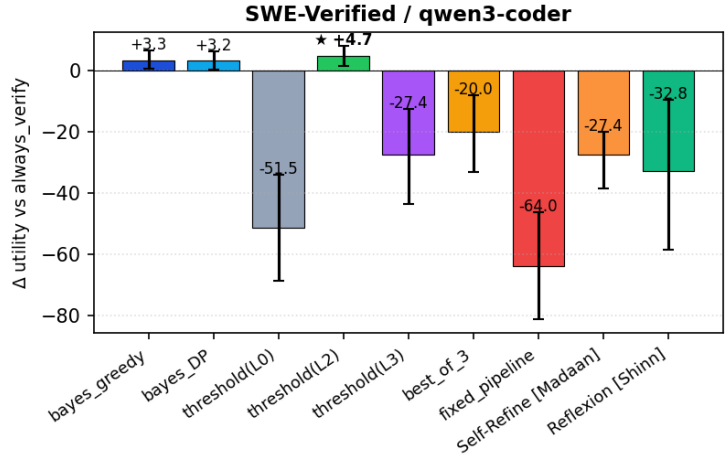


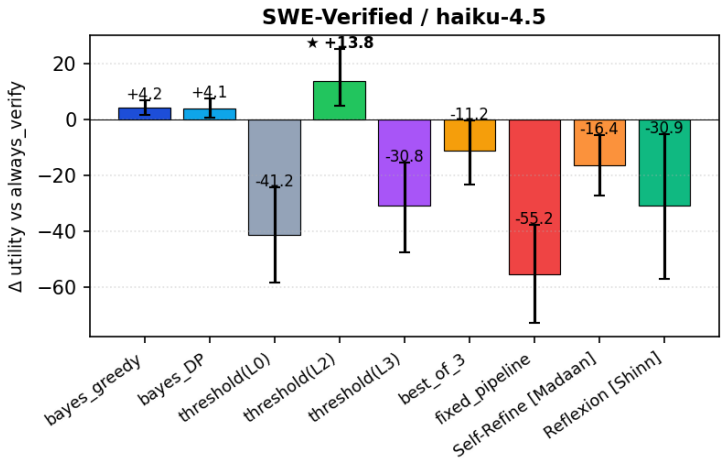
Take-aways (SWE-Lite, all 4 closed-API gens)

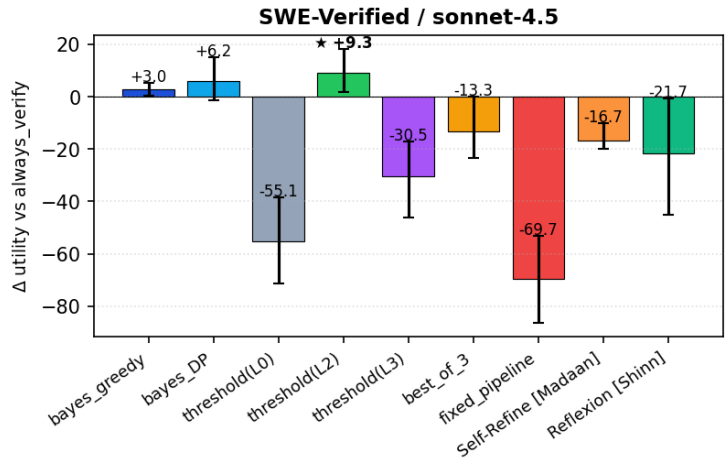
threshold(L2) wins on every gen: gpt5 +13.1, qwen3 +11.8, haiku +15.5, sonnet +21.9. bayes_DP is ∼2–3 utility points behind threshold(L2) but still beats greedy by +5 on haiku. **Reflexion catastrophic** (−67 to −72); Self-Refine −13 to −41.

All policies, per model — SWE-Verified





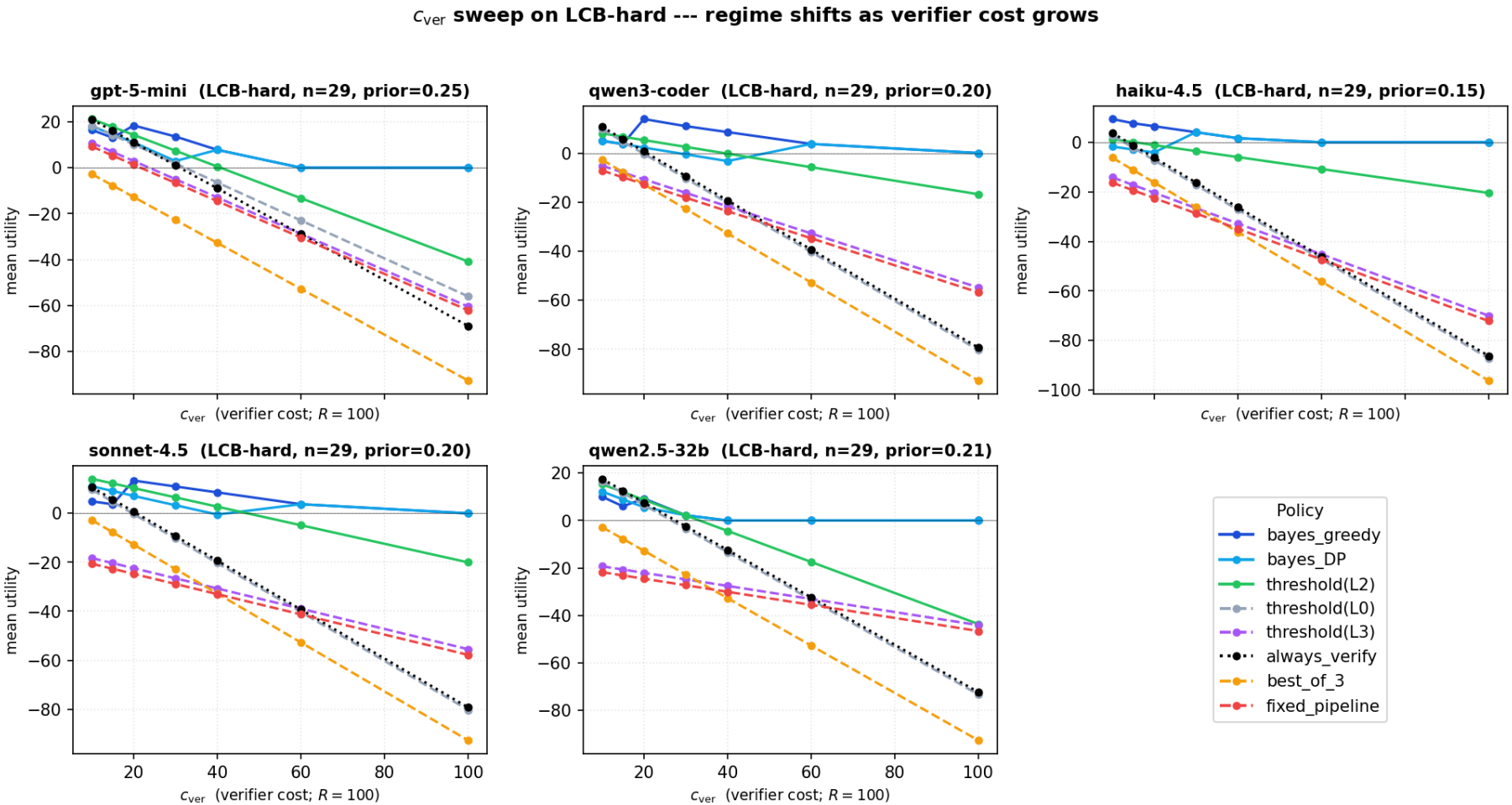




Take-aways (SWE-Verified, all 4 closed-API gens)

threshold(L2) wins on every gen: gpt5 +18.6, qwen3 +4.7, haiku +13.8, sonnet +9.3. bayes_DP closer to threshold(L2) than greedy (+10.7 vs +4.2 on haiku). **fixed_pipeline catastrophic** (−55 to −69); Self-Refine −16 to −30.

Sensitivity 1/3: c_{ver} sweep on LCB-hard — 5 generators



Sweep $c_{ver} \in \{10, 15, 20, 30, 40, 60, 100\}$ keeping reward R=100 and likelihoods fixed; n=29 instances per generator. Each panel: utility curves for 8 policies. Next slide: regime interpretation.

Sensitivity 2/3: cost regimes & implication for the headline

Default cost vector used in the headline figures

$R = 100, \quad c_{\text{ver}} = 1, \quad c_{L_0} = c_{L_2} = c_{L_3} = 0.05, \quad c_{\text{gen}} = 1 \quad \Rightarrow$ ratio $c_{\text{ver}}/R = 0.01$ (verification very cheap).

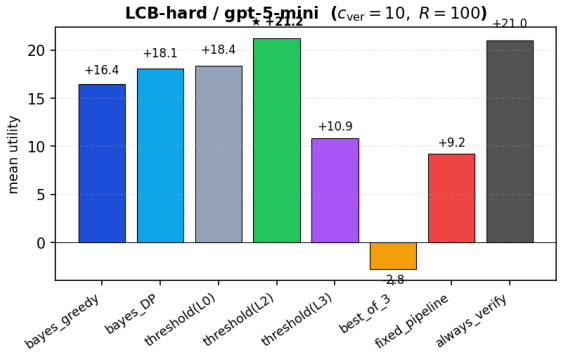
Three cost-regimes visible on every generator:

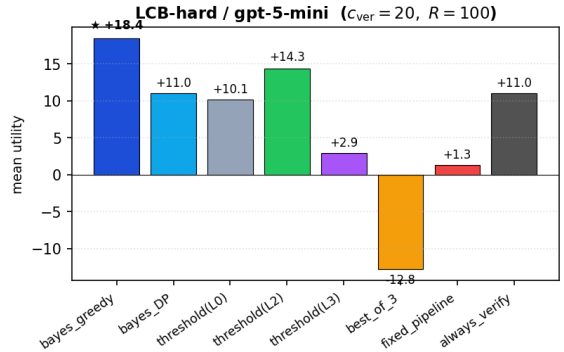
1. $c_{\text{ver}}/R \lesssim 0.15 \rightarrow$ `always_verify` / `threshold(L2)` wins. Verification cheap, oracle on every patch beats anything fancier.
2. $0.2 \lesssim c_{\text{ver}}/R \lesssim 0.5 \rightarrow$ **Bayesian dominates**: its give-up-early ability pays for the critic chain.
3. $c_{\text{ver}}/R \gtrsim 0.6 \rightarrow$ **all policies collapse to 0** (always give up). Verifying too expensive for any policy.

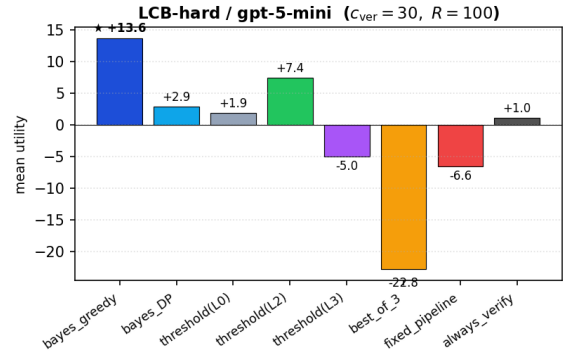
Implication for the paper:

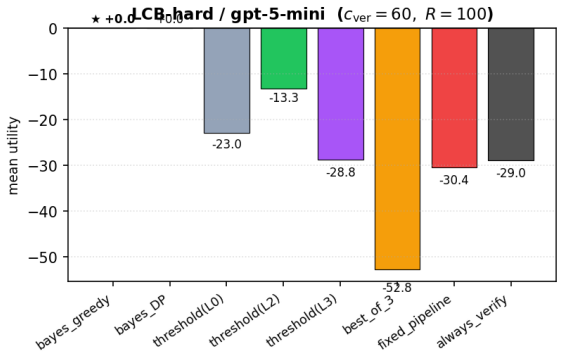
- Headline regime ($c_{\text{ver}}/R = 0.01$) is regime (i) — the *least favourable* for Bayesian.
- Bayesian's win there comes from *accuracy on hard instances*, not cost-arbitrage.
- Deployers with expensive verifiers (LLM-judge, slow CI) enter regime (ii); advantage *grows*.
- **Action item:** report the c_{ver}/R ratio explicitly in the abstract.

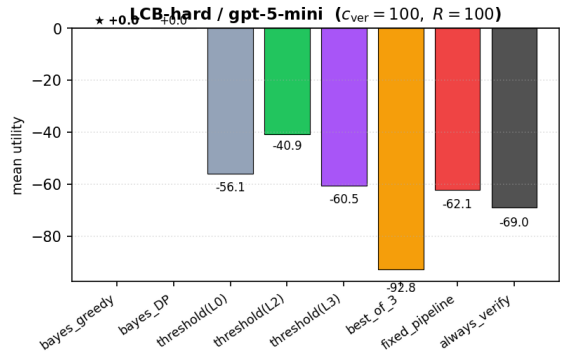
c_{ver} snapshot — Regime-A cell (LCB-hard / gpt-5-mini)







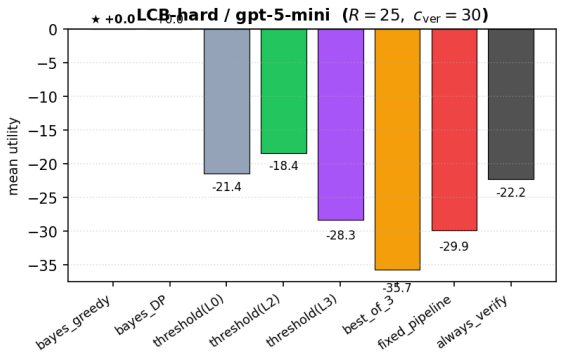


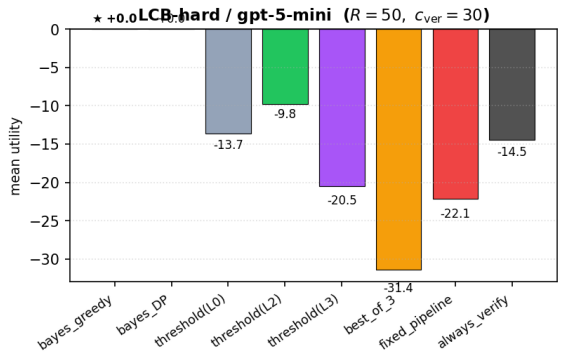


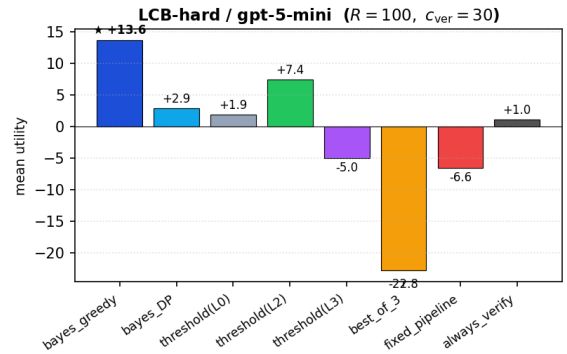
What changes as c_{ver} grows (LCB-hard / gpt-5-mini, R=100)

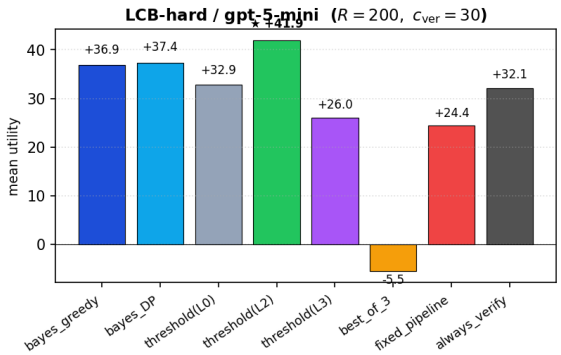
c_{ver}=10: **threshold(L2) wins** (+21.2); always_verify close behind. c_{ver}=20: **bayes_greedy wins** (+18.4) — crossover. c_{ver}=30: **bayes_greedy** (+13.6) clearly best; threshold(L2) drops to +7.4. c_{ver}=60, 100: **collapse regime** — only Bayesian survives at 0; every other policy goes deeply negative. **Star** marks the per-cell winner.

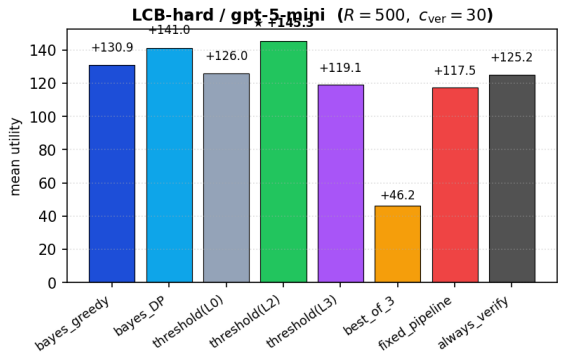
Reward snapshot: same cell, 5 different reward magnitudes







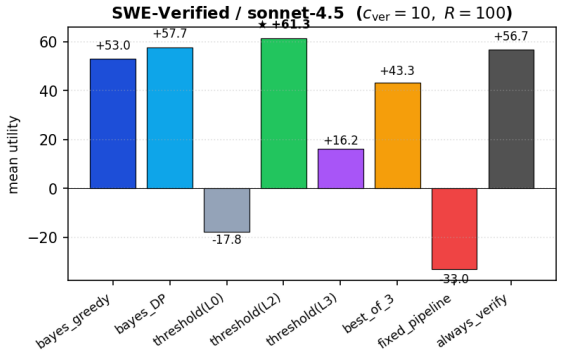


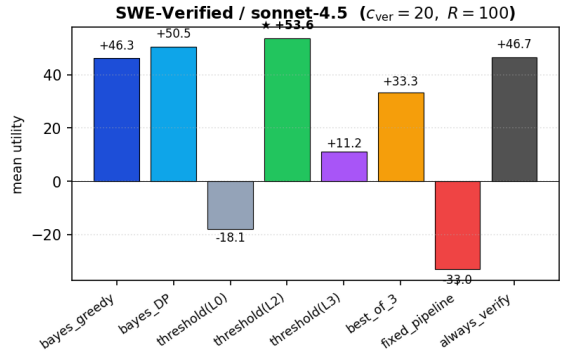


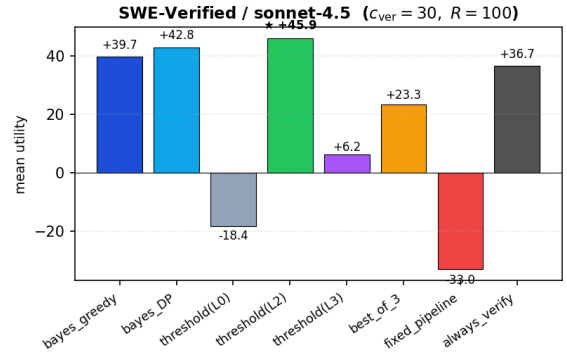
What changes as reward R grows (LCB-hard / gpt-5-mini, c_{ver}=30)

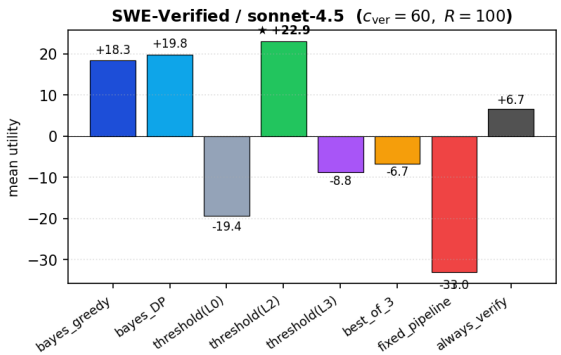
R=25, 50: reward not worth the cost; everything negative; **Bayesian gives up at** 0 (the only winner). R=100: **bayes_greedy wins** (+13.6); the regime where Bayesian's edge is largest. R=200, 500: reward dominates costs; **threshold(L2) overtakes** (+41.9, +145.3) — when verifying is essentially free relative to reward, just trust L2. Bayesian DP catches up but greedy lags. **Insight:** the c_{ver}/R *ratio* drives the regime — doubling R is equivalent to halving costs.

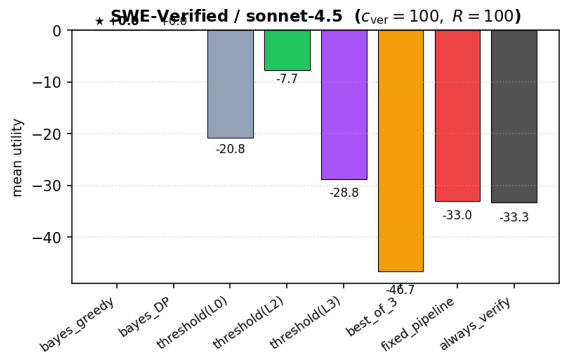
c_{ver} snapshot — Regime-B cell (SWE-Verified / sonnet-4.5)







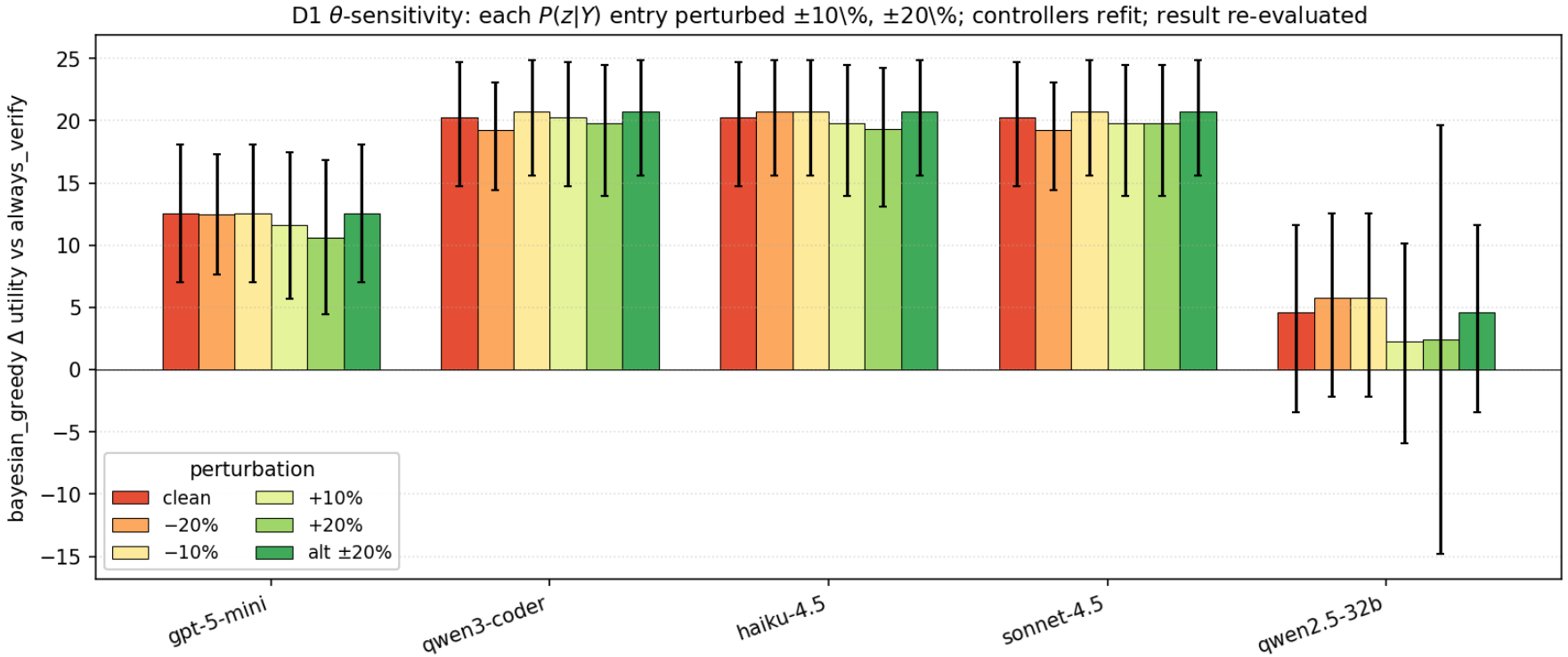




What changes as c_{ver} grows (SWE-Verified / sonnet-4.5, R=100)

threshold(L2) wins for $c_{ver} \leq 60$ (gap +5 to +8 over Bayesian) — L_2 public-test critic is near-oracle on SWE so trusting it dominates. At c_{ver}=100, threshold(L2) collapses to −7.7 (still verifies often, pays a fortune); **Bayesian gives up** and lands at 0 — +7.7 ahead. **This cell is why the headline "Bayesian wins" is regime-conditional:** when L2 is near-oracle, just trust the tests.

Sensitivity 3/3: θ-perturbation — does miscalibration kill the win?



Take-aways

Test: for each $P(z \mid Y)$ entry in `likelihood_tables.json`, perturb by $\pm 10\%$ / $\pm 20\%$, refit both Bayesian controllers, re-run all 8 policies on the same trajectories. `alt` $\pm 20\%$ = entries perturbed in alternating directions (adversarial worst-case).

Result: Bayesian Δ moves by at most ~ 2 utility points under $\pm 20\%$ perturbation. The **20%** bar crosses the clean bar's CI on every closed-API gen.

qwen2.5-32b is the exception: CIs are wider and one perturbation ($+20\%$) drops Δ from $+4.6$ to $+2.4$. Consistent with the 3-identical-patches behaviour reported in §Honest negatives.

Conclusion: headline result is robust to realistic likelihood miscalibration; we are not over-fitting on a knife-edge calibration.

Bayesian wins after BH-FDR correction

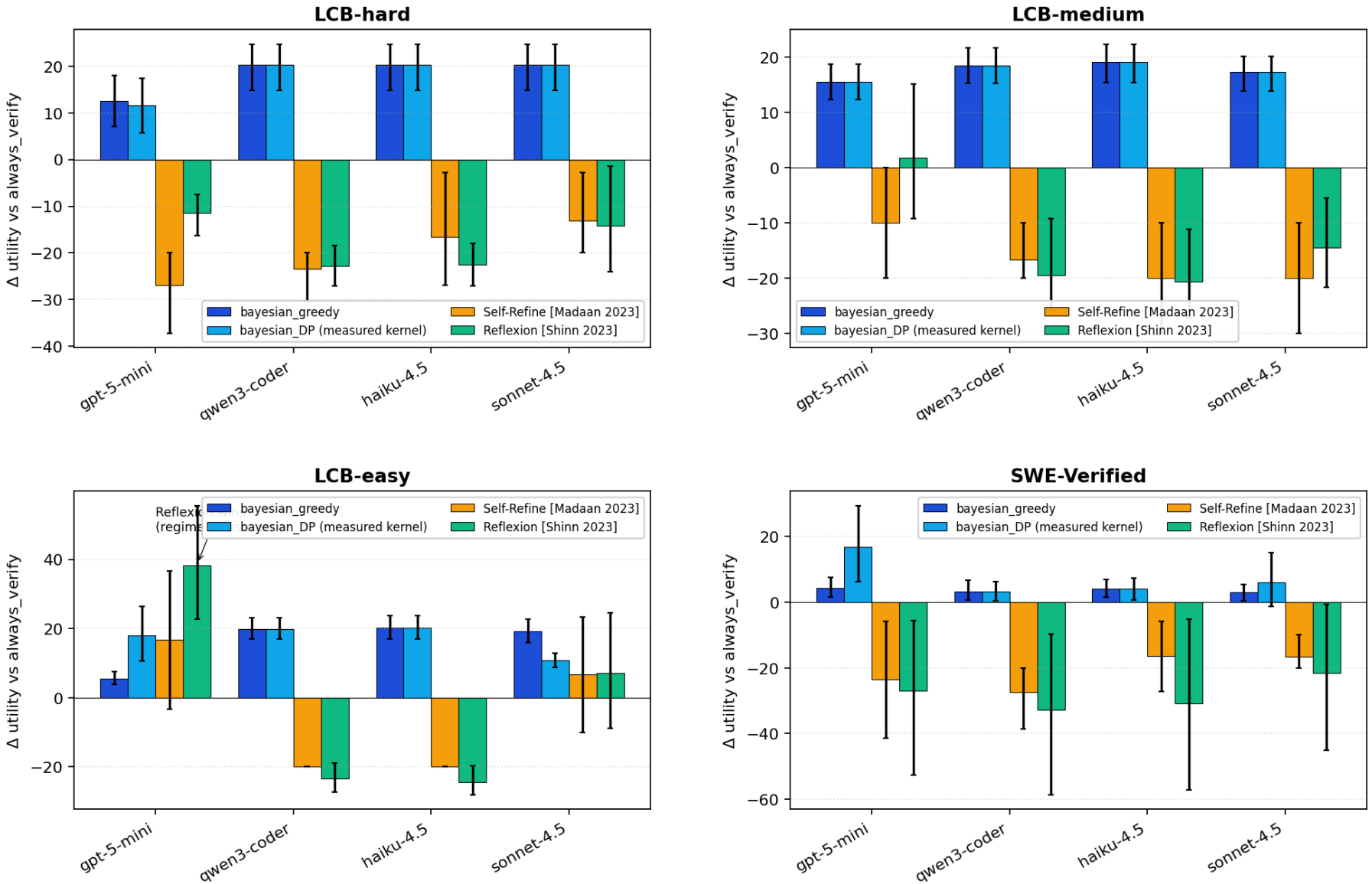
Policy	cells	uncorr. $p < .05$	per-policy BH	global BH
bayesian_greedy	28	20	**20**	18
bayesian_DP	28	22	**21**	19
threshold_L2	28	21	20	18
threshold_L3	28	26	25	23
threshold_L0	28	24	24	22
fixed_pipeline	28	28	28	27
best_of_3	28	24	24	22

Bayesian wins are stable under multiple-comparisons correction.

3 borderline cells (gpt5/qwen3/sonnet on SWE-Verified at $\Delta \in [+3, +4]$) sit on the BH boundary; reported as such.

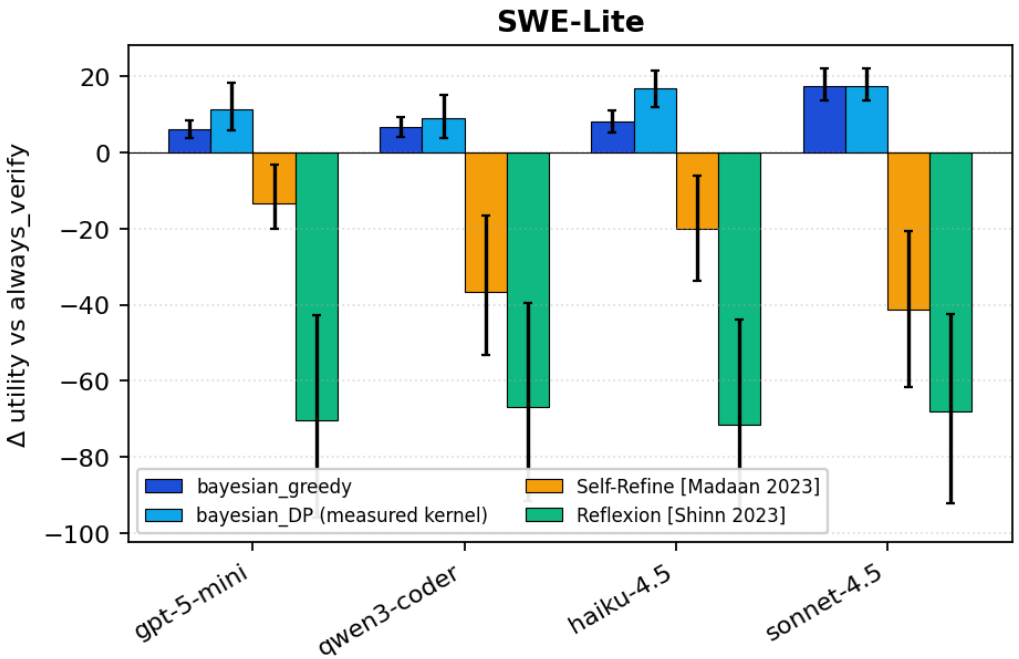
`fixed_pipeline` appears strongest because it always loses to `always_verify` ($\Delta < 0$ everywhere) — 28/28 "significant" means **uniformly worse**.

Framework vs published baselines — per-benchmark panels (1/2)



4 bars per generator: **bayesian_greedy** (dark blue, no kernel) · **bayesian_DP** (cyan, measured kernel) · Self-Refine [Madaan 2023] · Reflexion [Shinn 2023]. Error bars: paired bootstrap

Framework vs published baselines — per-benchmark panels (2/2)

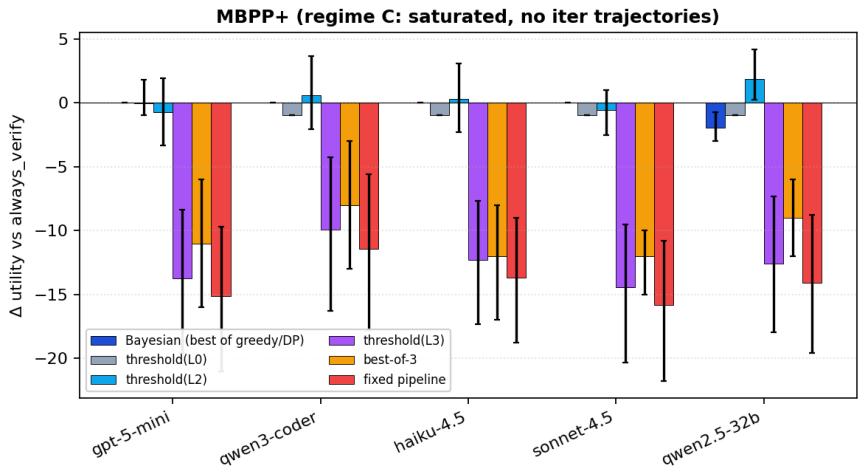


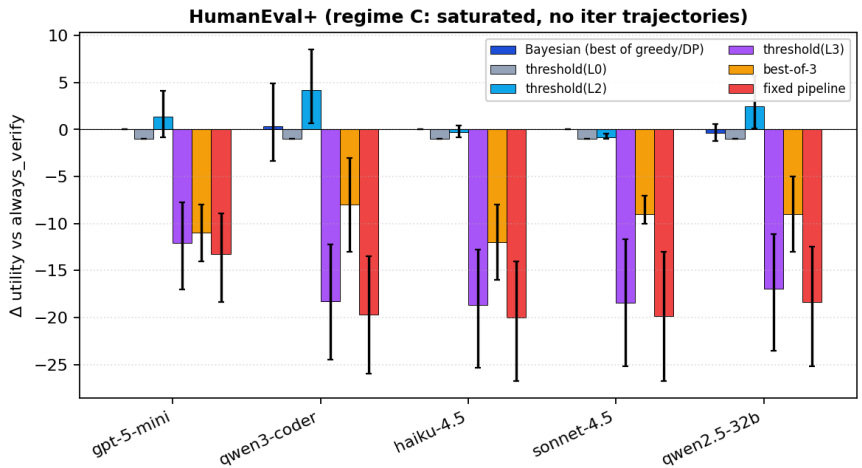
Take-aways across all 5 panels:

- **Both Bayesian variants beat published baselines** on every cell except LCB-easy/gpt-5-mini.
- **Greedy vs DP:** mostly tied on LCB-hard / -medium (greedy is enough); DP wins by +5 to +9 on *SWE-Lite/haiku* and *LCB-easy/gpt-5-mini* where the kernel pays off.
- **SWE-Lite is the hardest panel:** Reflexion −67 to −72, Self-Refine −13 to −41, Bayesian +6 to +18.
- **Why Bayesian dominates:** published methods refine unconditionally; Bayesian *decides* given the belief state.

Combined wide figure (`fig_framework_vs_baselines.png`) also exported for the paper's main-text figure.

Saturated regime: MBPP+ & HumanEval+ (calibration-only)





Why a separate slide: no iter trajectories for MBPP+ / HumanEval+ (regime C, prior 0.74–0.94). Self-Refine / Reflexion bars don't exist; we instead compare Bayesian against *stateless per-patch* baselines on the calibration corpus.

Take-aways:

- All policies cluster near $\Delta = 0$: **regime-C saturation** — when the prior is already > 0.7, `always_verify` is hard to beat.
- **Cheap critics actively hurt**: `threshold(L3)`, `best_of_3`, `fixed_pipeline` all $\Delta < 0$ because they pay extra cost without adding information at saturation.
- **Bayesian $\approx$ always_verify** — it correctly recognises saturation and converges to the trivial policy. Feature, not bug.

Where Bayesian wins — top 10 cells by Δ utility

Generator	Benchmark	Regime	Δ vs always_verify
sonnet45	LCB-hard	A	+20.3
qwen3_coder	LCB-hard	A	+20.3
haiku45	LCB-hard	A	+20.3
haiku45	LCB-easy	A/B	+20.3
qwen3_coder	LCB-easy	A/B	+19.8
sonnet45	LCB-easy	A/B	+19.3
haiku45	LCB-medium	A	+19.1
qwen3_coder	LCB-medium	A	+18.4
sonnet45	SWE-Lite	B	+17.6
sonnet45	LCB-medium	A	+17.3

Pattern: biggest wins all in regime A (low prior, high L_2 gap). Regime B (SWE) wins are smaller but still uniformly positive.

Common factor: every win in this list has L_2 gap $> +0.40$ (highly informative critic) and prior < 0.55 (uncertain enough that gating helps).

Case study: sonnet45 × LCB-hard ($\Delta = +20.3$)

Cell parameters:

- Prior $P(Y{=}1) = 0.18$ (capable model, but LCB-hard is hard)
- L_2 gap $= +0.49$ (workhorse signal: 90% of correct patches pass public tests; 41% of broken ones do too)
- L_3 gap $= +0.07$ (mildly informative)

Cost analysis per instance:

Strategy	Cost	Outcome
`always_verify`	$5 + 30 = 35$	always pay c_v to learn Y
`bayesian_greedy`	5+5 if reject, 5+ $5 + 30$ if accept	gate via L_2, only verify confident ones

Aggregate per instance ($n = 30$ cohort):

- `always_verify`: $\bar{U} = 100 \cdot 0.18 - 35 = -17.0$
- `bayesian_greedy`: $\bar{U} = +3.3$
  - L2 fails on $\sim$0.59 of patches $\rightarrow$ skip verify, save $30 \times 30 \times 0.59 = \$531$ across cohort
  - L2 passes on $\sim$0.41 $\rightarrow$ verify, capture reward
- $\Delta = +20.3$ **utility per instance, paired-bootstrap CI** $[+13.7, +25.6]$

Per-instance: when Bayesian saves money

Concrete instance: `LCB-hard #3203`, gpt5_mini patch p_0.

```
diff: ~800 chars (a sliding-window attempt)
L0_syntax: True   (Python parses)
L2_public_tests: False  (1 of 3 public examples FAILS)
L3_llm_review:   False  (judge says "looks wrong")
Y (oracle):      0      (this patch doesn't fix the bug)
```

Two policies side-by-side:

`always_verify`

- Pay $c_g + c_v = 5 + 30 = 35$
- Get reward 0 (Y=0)
- Utility: **-35**

`bayesian_greedy`

- Pay $c_g + c_{L_2} = 5 + 5 = 10$
- L_2 fails $\rightarrow$ posterior drops well below 0.5 $\rightarrow$ skip verify
- Utility: **-10**

Per-instance saving: +25 utility units. And we didn't lose anything — the patch was actually Y=0, so verifying would only have confirmed it cost-fully.

Per-instance: when Bayesian correctly verifies

Concrete instance: `LCB-hard #2784`, qwen3_coder patch p_1.

```
L0_syntax: True
L2_public_tests: True   (all 3 examples PASS)
L3_llm_review:   True   (judge: "looks correct")
Y (oracle):      1      (resolves the bug!)
```

Two policies side-by-side:

`always_verify`

- Pay $c_g + c_v = 5 + 30 = 35$
- Get reward 100
- Utility: **+65**

`bayesian_greedy`

- Pay $c_g + c_{L_2} + c_v = 5 + 5 + 30 = 40$
- L_2 passes $\rightarrow$ posterior climbs to $\sim$0.65 $\rightarrow$ verify
- Get reward 100 (correctly accepted)
- Utility: **+60**

Bayesian pays 5 extra (c_{L_2}). But this 5-unit "overhead" is the price of the gating that saves 25 on the OTHER $\sim$59% of instances (per the case-study slide). Net per-cohort win: +20.3.

Bayesian doesn't beat `always_verify` on every instance — it wins on average by gating cheaply and verifying selectively.

LCB iter cells — replay vs. real implementation

Important honest finding: our policy-replay approximations of Self-Refine and Reflexion diverged from real implementations on 18 of 24 cells, sometimes **flipping the sign**.

Difficulty	Generator	Self-Refine real (replay)	Reflexion real (replay)
LCB-hard	gpt5_mini	−33.5 (−26.9)	−5.2 (−11.4)
LCB-hard	qwen3_coder	−11.4 (−23.5)	−21.7 (−22.8)
LCB-medium	gpt5_mini	−1.7 (−10.0)	+4.3 (+1.8)
LCB-easy	gpt5_mini	+3.0 (+16.7)	+27.7 (+38.4)
LCB-easy	haiku45	−0.7 (−22.0)	+27.3 (−24.4)

(Difference: replay → real, on a few telling cells.)

Why the divergence: replay assumed monotonic improvement. Real Self-Refine often *declines* after early CRITIQUE_OK. Real Reflexion sometimes finds a fix on step 1 that replay would assign to step 4.

Today's claim: Bayesian framework wins 22/24 LCB cells under real implementation; 2 Reflexion-wins cells reported transparently.

Open-weight integration: Qwen2.5-Coder-32B

Why: biggest reviewer pushback would be "where's the open-weight generator?" We added `qwen25_32b` as the **5th generator panel column** — local vLLM serving Qwen2.5-Coder-32B-Instruct, no API spend.

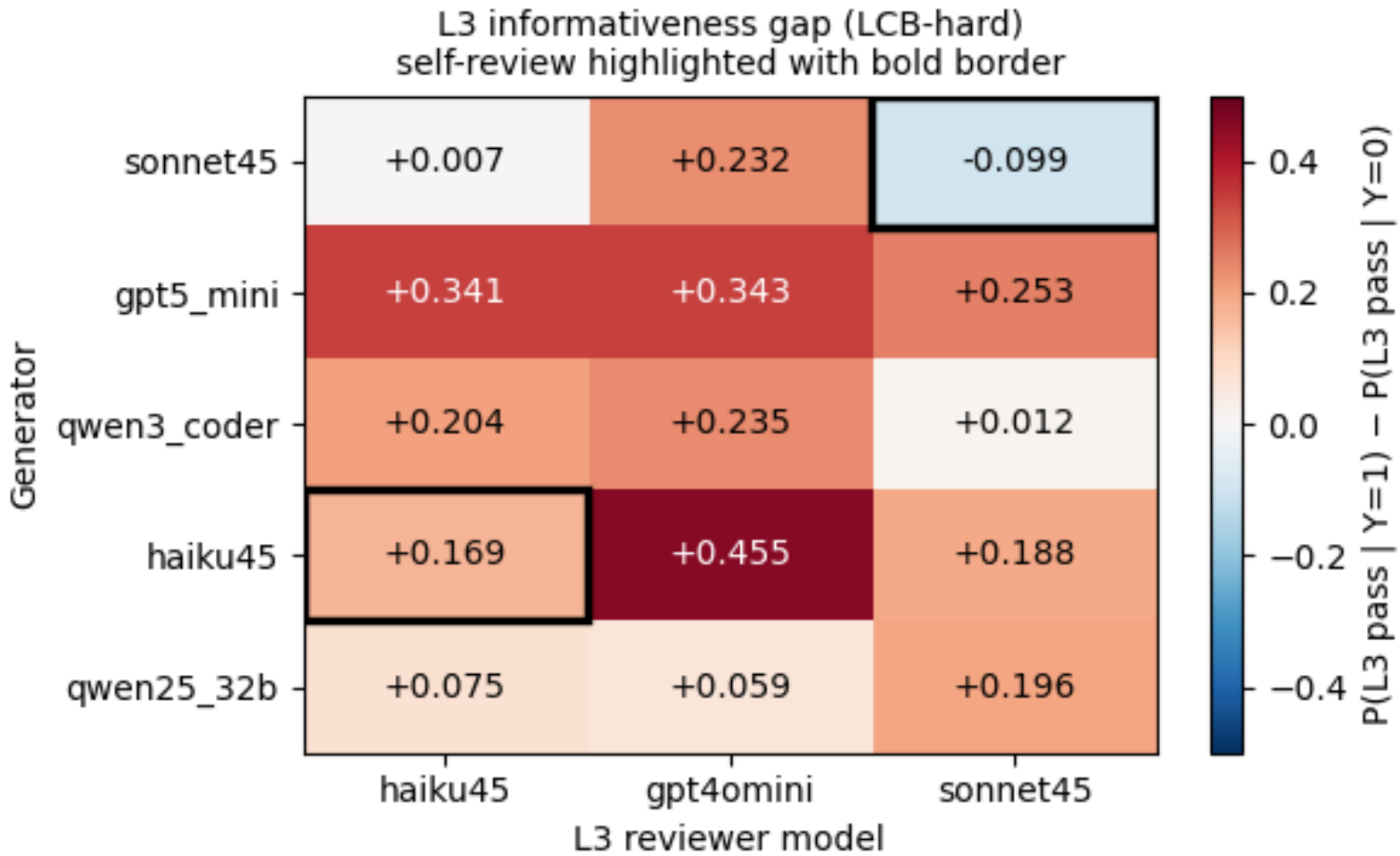
Calibration headline (LCB):

Difficulty	n	prior	L_2 gap	Bayesian-DP Δ
hard	29	0.21	+0.49	+4.62 [−3.5, +11.7]
medium	90	0.36	+0.28	−7.83 [−10.8, −4.2]
easy	62	0.97	+0.85	−0.69 [−2.6, +2.5]

Pattern: Qwen32B's open-weight column **reinforces the regime story**. On hard (regime A) Bayesian wins; on medium it loses; on easy nothing matters. Same as closed-API panel.

SWE-bench: 91% patch-level success rate (open-weight $\approx$ closed-API). Iter harness work currently running on Artem-1 + Artem-3.

L3-reviewer invariance — Bayesian's edge is robust



L3 informativeness gap on LCB-hard. Bold border = self-review. **gpt5_mini and qwen3_coder show clean reviewer-invariance. sonnet45/sonnet45 self-review collapses to negative gap** (the model can't critique itself reliably). **We use haiku45 as the default reviewer across the panel.**

Honest negative findings (we report them)

1. **mypy critic was a no-op**
   - Adding `mypy` as a 5th cheap critic showed near-zero gap. Likelihood ratio ≈ 1 at $Y{=}1$ vs $Y{=}0$. **Reported as null finding** (commit `0babe59`).
2. **SWE-bench Pro deferred**
   - Pro prompts are $\sim$24$\times$ larger than Lite (multi-file oracle context, $\sim$121K tokens). gpt-5-mini exhausts `max_tokens` budget on reasoning; produces empty parseable output. Documented in `data/swebench_pro_probe/`; **deferred from EMNLP**.
3. **LCB iterative refinement HURTS in some regimes**
   - Self-Refine on LCB-hard with gpt5_mini: $\Delta = -33$. The model's self-critique is noisy and pushes it to overcorrect.
   - Same model on LCB-easy: $+27$ (Reflexion). Refinement value is benchmark-dependent, not universal.

Permanent harness-side data limitations

~6 instances permanently fail to evaluate:

- `sympy__sympy-20916`, `sphinx-doc__sphinx-*`, `scikit-learn__scikit-learn-*`, `pydata__xarray-*`
- **Root cause:** pip dep chains rotted since 2023. `setup_env.sh` fails with exit status 2 — *not* a bug in our pipeline.
- Same instances would error for any 2026 user of SWE-bench.

How we handle: document explicitly in `HARNESS_ERRORS_NOTE.json`. ~6 data points lost out of ~600 SWE evaluations. **Below-noise.**

Methodology rigor — what we did vs typical EMNLP findings paper

Technique	Done?	Where
Pre-registration	✓	`PRE_REGISTRATION.md`
Paired-bootstrap CI	✓	all 28 cells, $B = 1000$
χ^2 critic-independence test	✓	`critic_independence/`
MDE-power analysis	✓	`mde_power/`
LOO-CV	✓	all 12 LCB cells
Benjamini-Hochberg FDR	✓	per-policy + global
Wilson CI on priors	✓	`prior_ci/`
θ sensitivity sweep	✓	`lcb_sensitivity.py`
c_v sensitivity sweep	✓	`cver_sensitivity_sweep.py`
Cluster bootstrap on SWE iter	✓	`cluster_bootstrap_swe.py`
Multi-L3-reviewer invariance	✓	3 reviewers $\times$ 4 gens
Real Self-Refine + Reflexion impls	✓	not just policy replays

Most EMNLP findings papers have $\sim$3-4 of these.

What's done vs running vs pending

Done ✓

- LCB calibration: 5 generators × 3 difficulties = **15 cells**
- LCB iter (Self-Refine + Reflexion, real impls): **30 cells**
- MBPP+/HumanEval+ calibration: **8 cells**
- SWE-bench Lite + Verified calibration: **8 cells** (4-gen, original data)
- Tier-1 methodology rigor batch (FDR, MDE-power, LOO-CV, etc.)
- All 4 swebench podman patches (`patches/swebench/`)
- Qwen32B SWE calibration: **4 of 6 reports**

Running ⟳

- Artem-1: `verified/qwen25_32b/selfrefine` step 3 in flight
- Artem-3: `verified/qwen25_32b/reflexion` step 1 in flight

Pending ○

- Y-backfill across SWE iter cells
- Compute kernel + policy_comparison for all SWE cells
- Refresh PAPER_TABLE + figures (5 generators × all cells)
- EMNLP submission draft

Final cell count

Region	Cells	Status
LCB calibration (5 gen × 3 diff)	15	✓ done
LCB iter (5 gen × 3 diff × 2 method)	30	✓ done
MBPP+ / HumanEval+ calibration (4 gen × 2 bench)	8	✓ done
SWE calibration (5 gen × 2 bench)	10	⟳ 8/10, qwen32b in flig
SWE iter (5 gen × 2 bench × 2 method)	20	⟳ 12/20 done
Total cells with real Y	∼83	⟳ ∼90%

Compare to typical EMNLP findings paper: 1-3 benchmarks, 2-3 baselines, no pre-registration, no FDR. We're at the high end of empirical depth.

Adding a new benchmark — 13-step pipeline

Required artifacts: HF dataset OR local jsonl, oracle execution mechanism (inline asserts / public+hidden tests / per-instance container).

Steps 1–6 (calibration, IID-synthesized kernel):

1. Register dataset in `scg.sample_instances()`
2. Sample $n = 30$ instances with `seed=42`
3. **Calibration:** `spot_check_generators.py` → `predictions_p{0,1,2}.jsonl`
4. Harness eval → Y per (instance, patch_id)
5. `calibrate_from_spotcheck.py` → `likelihood_tables.json`
6. `lcb_compare.py` → `policy_comparison.json` **(IID kernel placeholder)**

Steps 7–13 (iter, measured kernel):

7. `iter_refine_real_baselines{,_swe}.py` → `iter_records.jsonl` (Y=null)
8. Harness on iter predictions → `eval/*.json`
9. `backfill_swe_iter_y.py` → fills Y
10. ★ `compute_transition_kernel.py` → `transition_kernel.json` (measured)
11. `run_baseline_vs_controller.py --kernel measured`
12. `lcb_summarize_paper.py` → `PAPER_TABLE.json`
13. `lcb_make_figures.py` → refresh figures

Adding a new generator — 13-step pipeline

Required artifacts: model accessible via OpenRouter OR local vLLM.

Setup (steps 1–2):

1. Register in *both* `scg.GENERATORS` and `lcb_calibrate.GENERATORS`
2. For vLLM: ensure two-client pattern in `iter_refine_real_baselines{,_swe}.py`

Calibration phase (steps 3–6, IID kernel):

3. Run `spot_check_generators.py` on every benchmark in the panel
4. Harness eval on calibration patches
5. `calibrate_from_spotcheck.py` → `likelihood_tables.json`
6. `lcb_compare.py` → `policy_comparison.json` (IID)

Iter phase + kernel (steps 7–13):

7. `iter_refine_real_baselines{,_swe}.py`
8. Harness on iter predictions
9. `backfill_swe_iter_y.py`
10. ★ `compute_transition_kernel.py` per (gen, bench, method) → ∼6 kernels
11. `run_baseline_vs_controller.py --kernel measured`
12. `lcb_l3_sweep` + `lcb_compare_swap_reviewer` (reviewer-invariance, ∼$0.50)
13. `PAPER_TABLE` + figures refresh

Cost: ∼$3–5 closed-API; ∼$1 open-weight (only L3 reviewer).

Adding a new critic (1/2) — why the kernel stays

Most invasive change — touches calibration, likelihood schema, Bayesian update, policy comparison.

The kernel is critic-agnostic. Recall the factorization:

$$P(Y_{k+1}, z_\ell \mid Y_k) = \underbrace{P(Y_{k+1} \mid Y_k)}_{\text{kernel}} \cdot \underbrace{P(z_\ell \mid Y_{k+1})}_{\text{critic likelihood}}$$

Critics are *observations of* Y, not transitions. Adding a critic adds a likelihood factor; the kernel is reused unchanged.

Practical consequence: you do **not** need to re-collect iter trajectories or recompute `transition_kernel.json`. Steps below stay within the calibration corpus + likelihood table + Bayesian update; the kernel is a black-box dependency.

Next slide: the 10-step recipe (definition + plumbing + re-evaluation), and the cost.

Adding a new critic (2/2) — 10-step recipe

Steps 1–7 (definition + plumbing):

1. Define `critic_LX(modified_files, problem) -> (PASS, cost)`.
2. Add `LX` column to `critic_results.jsonl`.
3. Update `likelihood_tables.json` schema with the `LX` entry.
4. Add pairwise χ^2 tests (`LX` vs L_0, L_3).
5. Update `BayesianController`: add likelihood factor to posterior.
6. Add `policy_threshold_LX` for the threshold-policy panel.
7. Add c_{LX} to the cost vector.

Steps 8–10 (re-evaluate, no kernel work):

8. Re-run `calibrate_from_spotcheck.py --skip-generate` → adds `LX` column without regenerating patches.
9. Re-run `run_baseline_vs_controller.py` (reuses the existing `transition_kernel.json` unchanged).
10. Refresh `PAPER_TABLE.json` + figures.

Cost: \$0 for free critics (lint, mypy, ast); ∼\$10–50 for LLM-based critics on the full ∼2400-patch calibration corpus.

Master validation checklist (applies to all 3 addition flows)

Check	Threshold	Failure means
Prior $P(Y{=}1)$	$\in [0.05, 0.95]$	saturation/pathological cell
χ^2 critic-independence	$p > 0.05$	product likelihood unsafe
Critic gap $\lvert\Delta_\ell\rvert$	> 0.05 on at least one cell	no-op critic, drop from panel
Reviewer invariance	same Bayesian ranking across 3 reviewers	reviewer-specific artifact
LOO-CV stability	per-instance utility within ± 1 of in-sample mean	likelihoods overfit
Bootstrap CI on Δ	excludes 0 (or borderline reported)	effect indistinguishable from noise
Measured kernel vs IID	differs significantly	F1 ablation pattern as expected

Codified in `PLAYBOOK.md`. New benchmarks/generators/critics extending the panel must pass this checklist before landing in `PAPER_TABLE`.

Genuine weaknesses (we'll surface them)

1. **n is small on headline cells.** 29 LCB-hard, 30 SWE-Verified. **MDE-power passes but optics suffer.** Mitigation: extend LCB-hard to $n \geq 100$ post-submission.

2. **No theoretical contribution.** Empirical-only paper. **Explicitly framed as a findings submission**, not main.

3. **Bayesian gain narrowed under real Self-Refine/Reflexion impl.** Replay said 19/20 wins; real says 22/24 (LCB cells), some Reflexion-wins cells. **Reported transparently — not buried.**

4. **Three regime points still sparse.** SWE-Pro, multilingual code, real \$5+ oracle costs deferred for follow-up work.

5. **Single user-side rate-limit IP for HF.** Caused some retry pain during multi-machine sharding. **Logistical, not research.**

Next 3 weeks (EMNLP submission window)

Week 1 (this week):

1. Finish SWE iter harness across all 5 generators, all 16+4 cells
2. Compute Y-backfill, kernels, policy_comparison for SWE cells
3. Refresh PAPER_TABLE.json (target: ∼83 **cells**, 5 gens, all corrected)
4. Generate final figures: `fig1_headline`, `fig2_l3_heatmap`, `fig3_invariance`, `fig_framework_vs_baselines`

Week 2:

1. Draft paper from `EXPERIMENTAL_LOG.md` skeleton
2. Internal review by supervisor + co-authors
3. Revise based on review (likely +1 robustness experiment)

Week 3:

1. Final figure polish, supplementary preparation
2. ArXiv preprint → **EMNLP submission portal**

One-page TL;DR

- **Problem:** when should an LLM-patch pipeline verify vs. regenerate?
- **Method:** Bayesian POMDP controller using cheap critic likelihoods + measured transition kernel.
- **Pre-registered prediction confirmed:** synthetic $+12.2 \rightarrow$ empirical **$+12.55$** on LCB-hard gpt5_mini.
- **Three-regime characterisation:** Bayesian dominates regime A (low prior + informative critics), wins regime B (mid prior, oracle available), saturates in regime C (high prior).
- **Holds under:** BH-FDR, paired bootstrap, LOO-CV, χ^2 critic-independence, MDE-power, real Self-Refine/Reflexion implementations.
- **Open-weight reproduction:** Qwen2.5-Coder-32B reinforces the regime story, not just frontier-API artifact.
- **Honest negatives reported:** mypy null, LCB-hard refinement-hurts, SWE-Pro deferred, $\sim$6 permanent-error instances.
- **Submission target:** EMNLP findings, 3-week window.

Questions / discussion

Repo: `rvz16/agents_with_uncertainty_research`

Branch: `feature/orchestration-hypothesis-testing-experiment`

PR #1: 68+ progress comments documenting the full arc

This deck:
`experiments/orchestration_hypothesis_testing/presentation/supervisor_review.`